

Vivekanand Education Society's Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai
Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.



Department of Information Technology

CERTIFICATE

This is to certify that Jai Navani of D15A semester VI, have successfully completed necessary experiments in the MAD & PWA Lab under my supervision in **VES Institute of Technology** during the academic year 2024-2025.

Lab Assistant

Subject Teacher

Mrs. Kajal Joseph

Principal

Head of Department

Dr. Mrs. Shalu Chopra

Name of the Course : MAD & PWA Lab**Course Code** : ITL604**Year/Sem/Class** : D15A/D15B**A.Y.:** 24-25**Faculty Incharge** : Mrs. Kajal Joseph.**Lab Teachers** : Mrs. Kajal Joseph.**Email** : kajal.jewani@ves.ac.in**Programme Outcomes:** The graduate will be able to:

PO1) Basic Engineering knowledge: An ability to apply the fundamental knowledge in mathematics, science and engineering to solve problems in Computer engineering.

PO2) Problem Analysis: Identify, formulate, research literature and analyze computer engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and computer engineering and sciences.

PO3) Design/ Development of Solutions: Design solutions for complex computer engineering problems and design system components or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal and environmental considerations.

PO4) Conduct investigations of complex engineering problems using research-based knowledge and research methods including design of experiments, analysis and interpretation of data and synthesis of information to provide valid conclusions.

PO5) Modern Tool Usage: Create, select and apply appropriate techniques, resources and modern computer engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6) The Engineer and Society: Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to computer engineering practice.

PO7) Environment and Sustainability: Understand the impact of professional computer engineering solutions in societal and environmental contexts and demonstrate knowledge of and need for sustainable development.

PO8) Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of computer engineering practice.

PO9) Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams and in multidisciplinary settings.

PO10) Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations and give and receive clear instructions.

PO11) Project Management and Finance: Demonstrate knowledge and understanding of computer engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12) Life-long Learning: Recognize the need for and have the preparation and ability to engage in independent and lifelong learning in the broadest context of technological change.

Program specific Outcomes

PSO1) An ability to manage and analyze data / information effectively for making better decisions.

PSO2) Demonstrate the ability to use state of the art technologies and tools including Free and Open Source Software (FOSS) tools in developing software.

Lab Objectives:

Sr. No.	Lab Objectives
The Lab experiments aims:	
1	Learn the basics of the Flutter framework.
2	Develop the App UI by incorporating widgets, layouts, gestures and animation
3	Create a production ready Flutter App by including files and firebase backend service.
4	Learn the Essential technologies, and Concepts of PWAs to get started as quickly and efficiently as possible
5	Develop responsive web applications by combining AJAX development techniques with the jQuery JavaScript library.
6	Understand how service workers operate and also learn to Test and Deploy PWA.

Lab Outcomes:

Sr. No.	Lab Outcomes	Cognitive levels of attainment as per Bloom's Taxonomy
On Completion of the course the learner/student should be able to:		
1	Understand cross platform mobile application development using Flutter framework	L1, L2
2	Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation	L3
3	Analyze and Build production ready Flutter App by incorporating backend services and deploying on Android / iOS	L3, L4
4	Understand various PWA frameworks and their requirements	L1, L2
5	Design and Develop a responsive User Interface by applying PWA Design techniques	L3
6	Develop and Analyse PWA Features and deploy it over app hosting solutions	L3, L4

Index

Sr. No	Experiment Title	LO	DOP	DOS	Grade
1.	To install and configure the Flutter Environment	LO1			
2.	To design Flutter UI by including common widgets.	LO2			
3.	To include icons, images, fonts in Flutter app	LO2			
4.	To create an interactive Form using form widget	LO2			
5.	To apply navigation, routing and gestures in Flutter App	LO2			
6.	To Connect Flutter UI with fireBase database	LO3			
7.	To write meta data of your Ecommerce PWA in a Web app manifest file to enable “add to homescreen feature”.	LO4			
8.	To code and register a service worker, and complete the install and activation process for a new service worker for the E-commerce PWA	LO5			
9.	To implement Service worker events like fetch, sync and push for E-commerce PWA	LO5			
10.	To study and implement deployment of Ecommerce PWA to GitHub Pages.	LO5			
11.	To use google Lighthouse PWA Analysis Tool to test the PWA functioning.	LO6			
12.	Assignment-1	LO1,LO2 ,LO3			
13.	Assignment-2	LO4,LO5 ,LO6			

MAD and PWA LAB

Name: Jai Navani

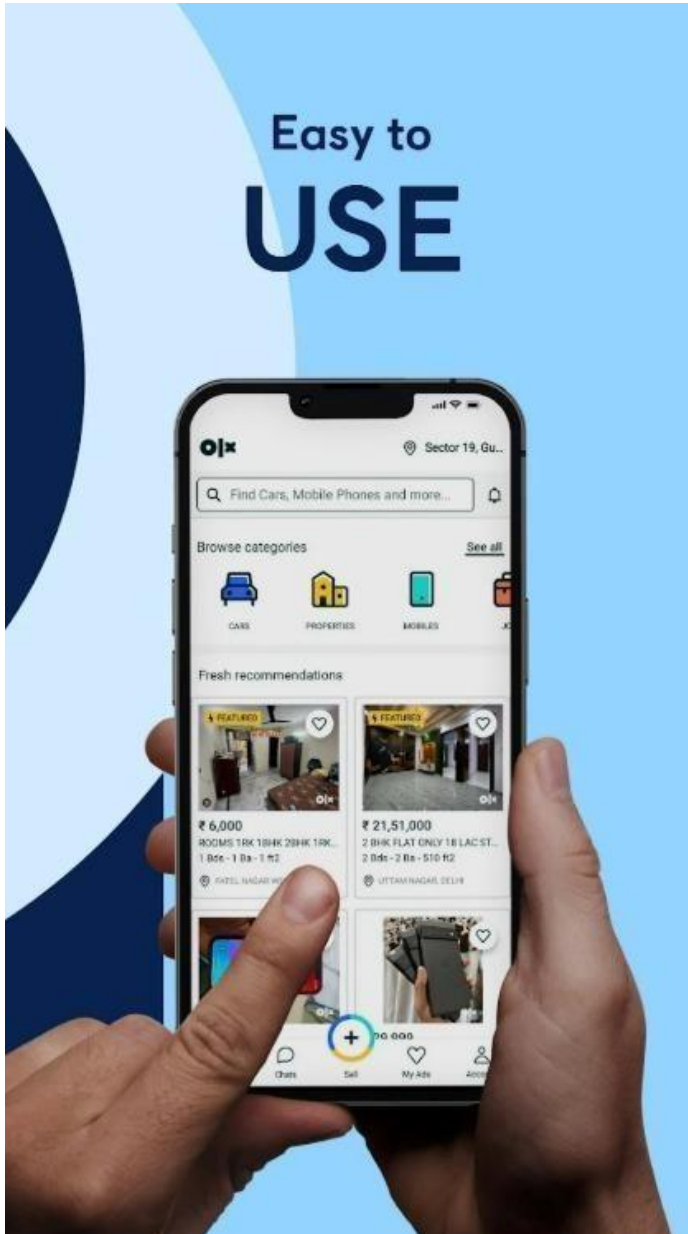
Class: D15A

Roll no:30

Aim: Selecting features for application development, the features should comprise of:

1. Common widgets
2. Should include icons, images, charts etc.
3. Should have an interactive Form
4. Should apply navigation, routing and gestures
5. Should connect with FireBase database

Screen Shot	Features
	Landing Page : 1. Sign in using email address 2. Firebase for authentication 3. Firebase : a. User : Username , email and password b. Score : Category and score c. Questions : Question , options, correct option



User homepage:

1. Product display
2. Navigation pages
3. Search

Easy to
BUY



Product page:

**Display and information page for the
single page**



Rohit



Lane stereo audio mixer p mx...

₹ 9,000

Friday, 20 Dec

Is it available?

11:42 pm

This ad has been disabled by the seller

Delete

Chatting feature :

Chat with the buyer

My Ads



Want to sell more?
Post more Ads for less. Packages that help you save money.

Show me packages

View all ads (2) ✓

FROM: 20 DEC 2024



Swiss military trolley bag
₹ 2,900

Views: - | Likes: -

NOT POSTED

Wait 25 days to post for free or pay to post now

Post Now

FROM: 20 DEC 2024 - TO: 19 JAN 2025



Lane stereo audio mixer p mx-602
₹ 9,000

 Home  Chats  Sell  My Ads  Account

Selling page:

Selling of the products

Project Title: OLX(buy&sell)

Roll No. 30

MAD & PWA Lab

Journal

Experiment No.	01
Experiment Title.	To install and configure the Flutter Environment
Roll No.	30
Name	Jai Navani
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO1: Understand cross platform mobile application development using Flutter framework
Grade:	

EXP 1: Installation and Configuration of Flutter Environment.

Install the Flutter SDK

Step 1: Download the installation bundle of the Flutter Software Development Kit for windows. To download Flutter SDK, Go to its official [website https://docs.flutter.dev/get-started/install](https://docs.flutter.dev/get-started/install) , you will get the following screen.

The screenshot displays the Flutter documentation website. At the top, there is a navigation bar with links for Multi-Platform, Development, Ecosystem, Showcase, Docs, and a search icon. A 'Get started' button is located in the top right corner. Below the navigation bar, a dark blue banner reads 'Celebrating Flutter's production era! Learn more' and 'Also, check out What's new on the website.' The main content area is titled 'Choose your development platform to get started' with a breadcrumb 'Get started > Install'. It features four large buttons for 'Windows Current device', 'macOS', 'Linux', and 'ChromeOS'. A light blue box below these buttons contains a note about 'Developing in China'. At the bottom of the main content area, a cookie notice from docs.flutter.dev is visible. On the left side, there is a sidebar with a list of links under 'Get started', including 'Set up Flutter', 'Learn Flutter', 'Stay up to date', 'App solutions', 'User interface', 'Introduction', 'Widget catalog', 'Layout', 'Adaptive & responsive design', and 'Design & theming'. Below the main content area, there is a section titled 'From another platform?' with links for 'Flutter for Android devs', 'Flutter for iOS devs', 'Flutter for React Native devs', 'Flutter for web devs', and 'Flutter for Xamarin.Forms devs'. A light blue box with a note about Chrome OS is also present, and a yellow box with an important note about China is at the bottom.

Flutter

Multi-Platform Development Ecosystem Showcase Docs

Celebrating Flutter's production era! Learn more
Also, check out What's new on the website.

Get started
Set up Flutter
Learn Flutter
Stay up to date
App solutions
User interface
Introduction
Widget catalog
Layout
Adaptive & responsive design
Design & theming

Choose your development platform to get started

Get started > Install

Windows Current device macOS Linux ChromeOS

Developing in China
If you want to use Flutter in China, check out using Flutter in China. If you're not developing in China, ignore this notice and follow the other instructions on this page.

docs.flutter.dev uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic. Learn more. **OK, got it**

Note: Are you on Chrome OS?
If so, see the official [Chrome OS Flutter installation docs!](#)

Windows macOS Linux

Important: If you're in China, first read [Using Flutter in China](#).

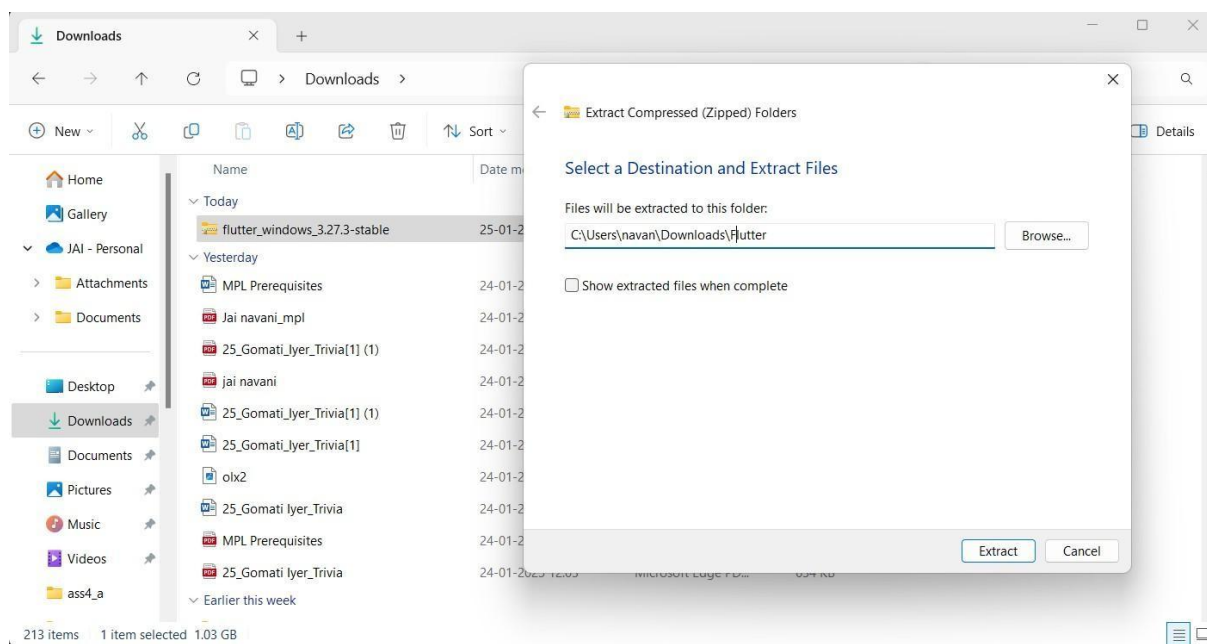
From another platform?
Flutter for Android devs
Flutter for iOS devs
Flutter for React Native devs
Flutter for web devs
Flutter for Xamarin.Forms devs

Step 2: Next, to download the latest Flutter SDK, click on the Windows **icon**. Here, you will find the download link for [SDK](#).

Step 3: When your download is complete, extract the **zip** file and place it in the desired installation folder or location, for example, C: /Flutter.

Step 4: To run the Flutter command in regular windows console, you need to update the system path to include the flutter bin directory. The following steps are required to do this:

Step 4.1: Go to MyComputer properties -> advanced tab -> environment variables. You will get the following screen



Step 4.3: In the above window, click on New->write path of Flutter bin folder in variable value -> ok -> ok -> ok.

Step 5: Now, run the \$ **flutter** command in command prompt.

```
Command Prompt - flutter - f X + v
Microsoft Windows [Version 10.0.22631.4602]
(c) Microsoft Corporation. All rights reserved.

C:\Users\navan>flutter
Manage your Flutter app development.

Common commands:

  flutter create <output directory>
    Create a new Flutter project in the specified directory.

  flutter run [options]
    Run your Flutter application on an attached device or in an emulator.

Usage: flutter <command> [arguments]

Global options:
-h, --help                Print this usage information.
-v, --verbose              Noisy logging, including all shell commands executed.
                           If used with "--help", shows hidden options. If used with "flutter doctor", shows additional
                           diagnostic information. (Use "-vv" to force verbose logging in those cases.)
-d, --device-id            Target device id or name (prefixes allowed).
--version                 Reports the version of this tool.
--enable-analytics         Enable telemetry reporting each time a flutter or dart command runs.
--disable-analytics        Disable telemetry reporting each time a flutter or dart command runs, until it is
                           re-enabled.
--suppress-analytics       Suppress analytics reporting for the current CLI invocation.
```

Now, run the \$ **flutter doctor** command. This command checks for all the requirements of Flutter app development and displays a report of the status of your Flutter installation.

```
Select Command Prompt
See Google's privacy policy:
https://policies.google.com/privacy

C:\Users\jalpa>
C:\Users\jalpa>
C:\Users\jalpa>flutter doctor
Running "flutter pub get" in flutter_tools... 17.0s
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 2.8.1, on Microsoft Windows [Version 10.0.19042.1415], locale en-US)
[✗] Android toolchain - develop for Android devices
     ✗ Unable to locate Android SDK.
        Install Android Studio from: https://developer.android.com/studio/index.html
        On first launch it will assist you in installing the Android SDK components.
        (or visit https://flutter.dev/docs/get-started/install/windows#android-setup for detailed instructions).
        If the Android SDK has been installed to a custom location, please use
        `flutter config --android-sdk` to update to that location.
[✓] Chrome - develop for the web
[!] Android Studio (not installed)
[✓] VS Code (version 1.55.2)
[✓] Connected device (2 available)

! Doctor found issues in 2 categories.

C:\Users\jalpa>flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 2.8.1, on Microsoft Windows [Version 10.0.19042.1415], locale en-US)
[!] Android toolchain - develop for Android devices (Android SDK version 32.0.0)
     ✗ cmdline-tools component is missing
       Run `path/to/sdkmanager --install "cmdline-tools;latest"`
       See https://developer.android.com/studio/command-line for more details.
     ✗ Android license status unknown.
       Run `flutter doctor --android-licenses` to accept the SDK licenses.
       See https://flutter.dev/docs/get-started/install/windows#android-setup for more details.
[✓] Chrome - develop for the web
[✓] Android Studio (version 2020.3)
[✓] VS Code (version 1.55.2)
[✓] Connected device (2 available)

! Doctor found issues in 1 category.
```

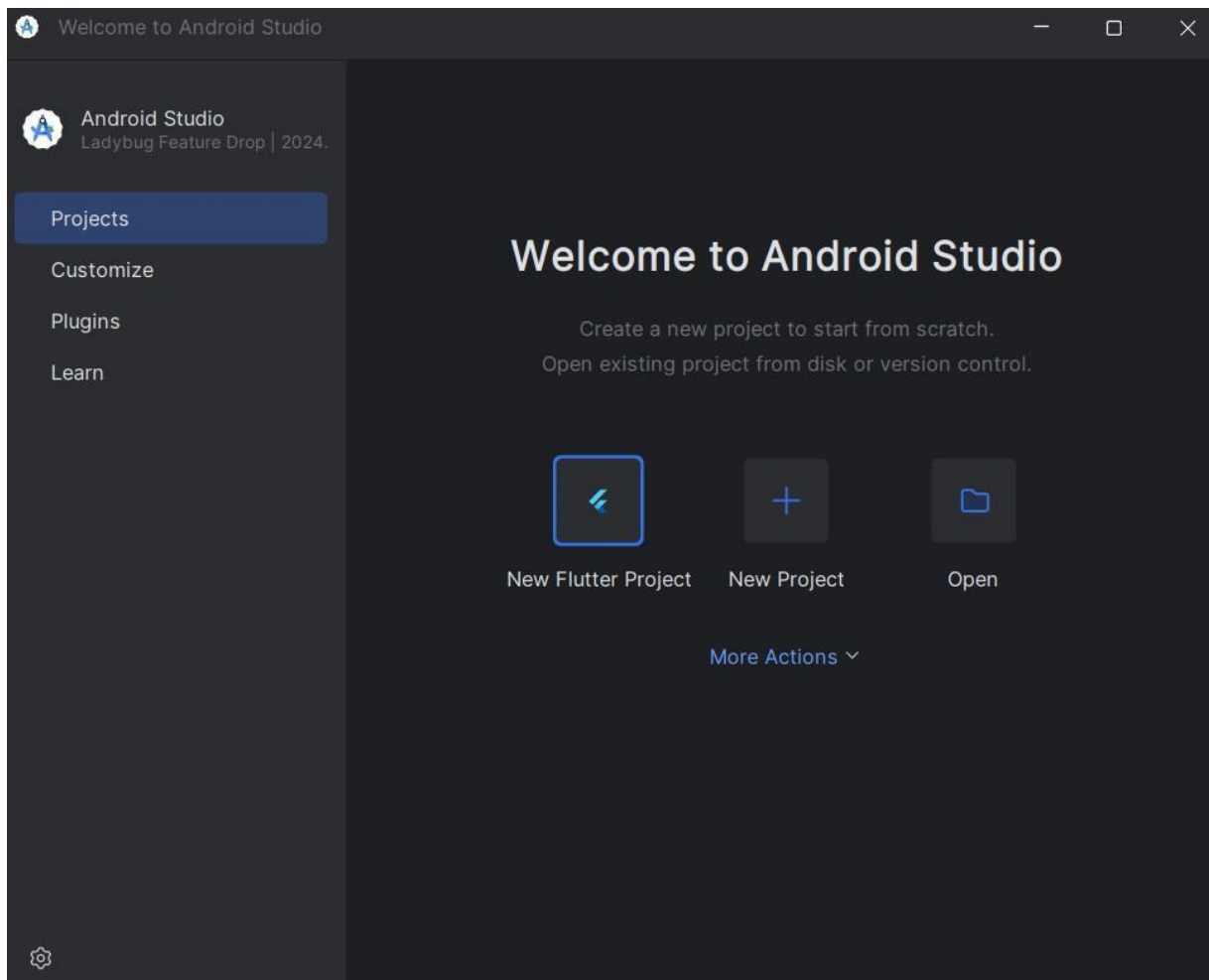
Step 6: When you run the above command, it will analyze the system and show its report, as shown in the below image. Here, you will find the details of all missing tools, which required to run Flutter as well as the development tools that are available but not connected with the device.

Step 7: Install the Android SDK. If the flutter doctor command does not find the Android SDK tool in your system, then you need first to install the Android Studio IDE. To install Android Studio IDE, do the following steps.

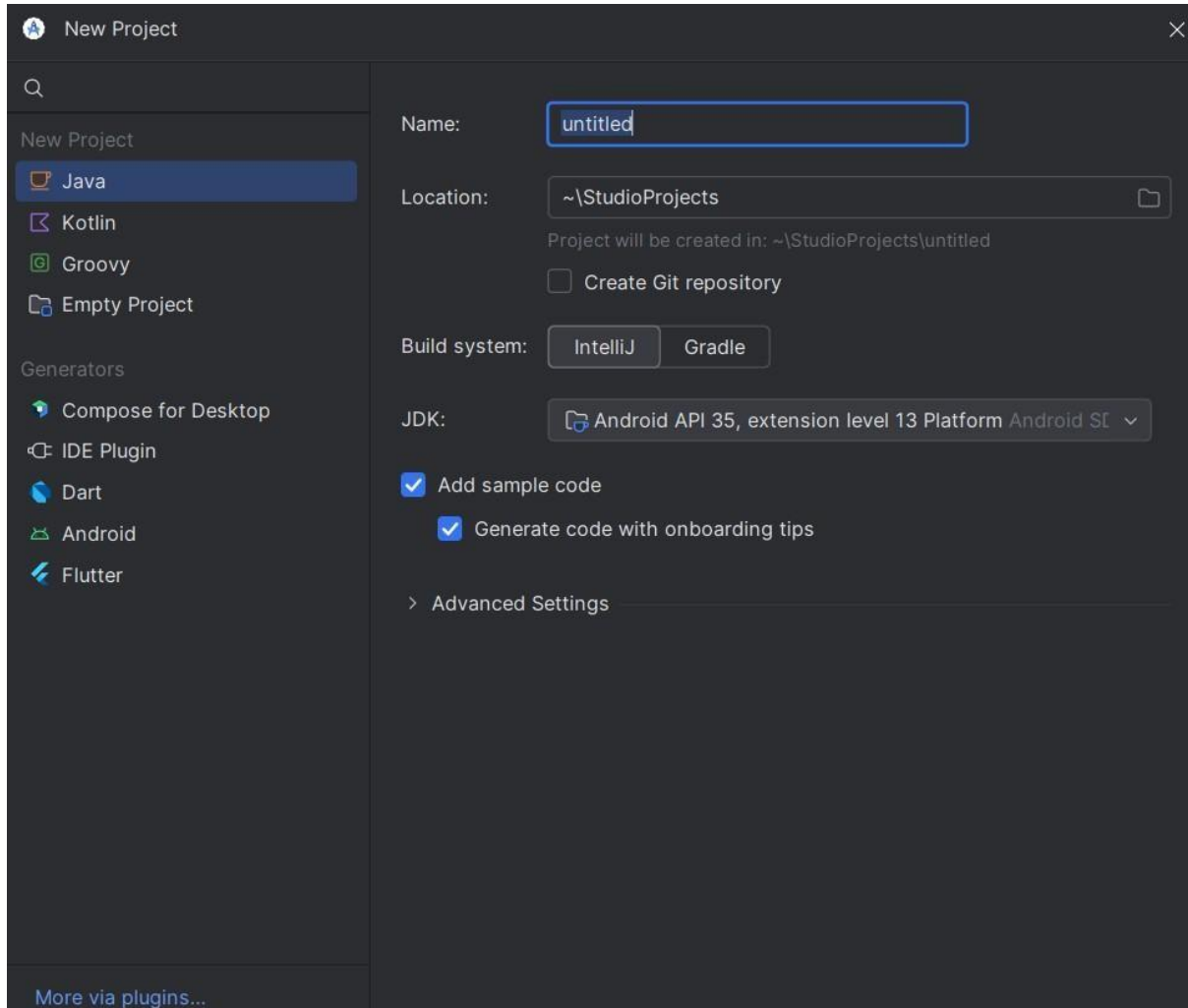
Step 7.1: Download the latest Android Studio executable or zip file from the [official site](#).

Step 7.2: When the download is complete, open the **.exe** file and run it. You will get the following dialog box.

After download:

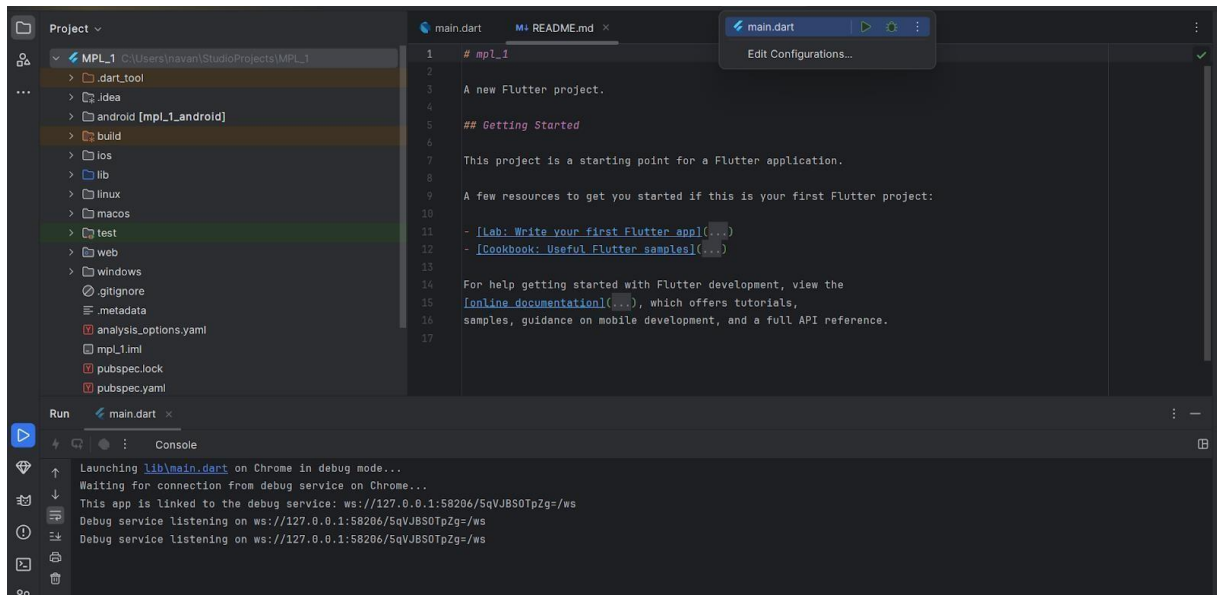


Step 7.4: In the above screen, click Next-> Finish. Once the Finish button is clicked, you need to choose the 'Don't import Settings option' and click OK. It will start the Android Studio.



Step 8: Next, you need to set up an Android emulator. It is responsible for running and testing the Flutter application.

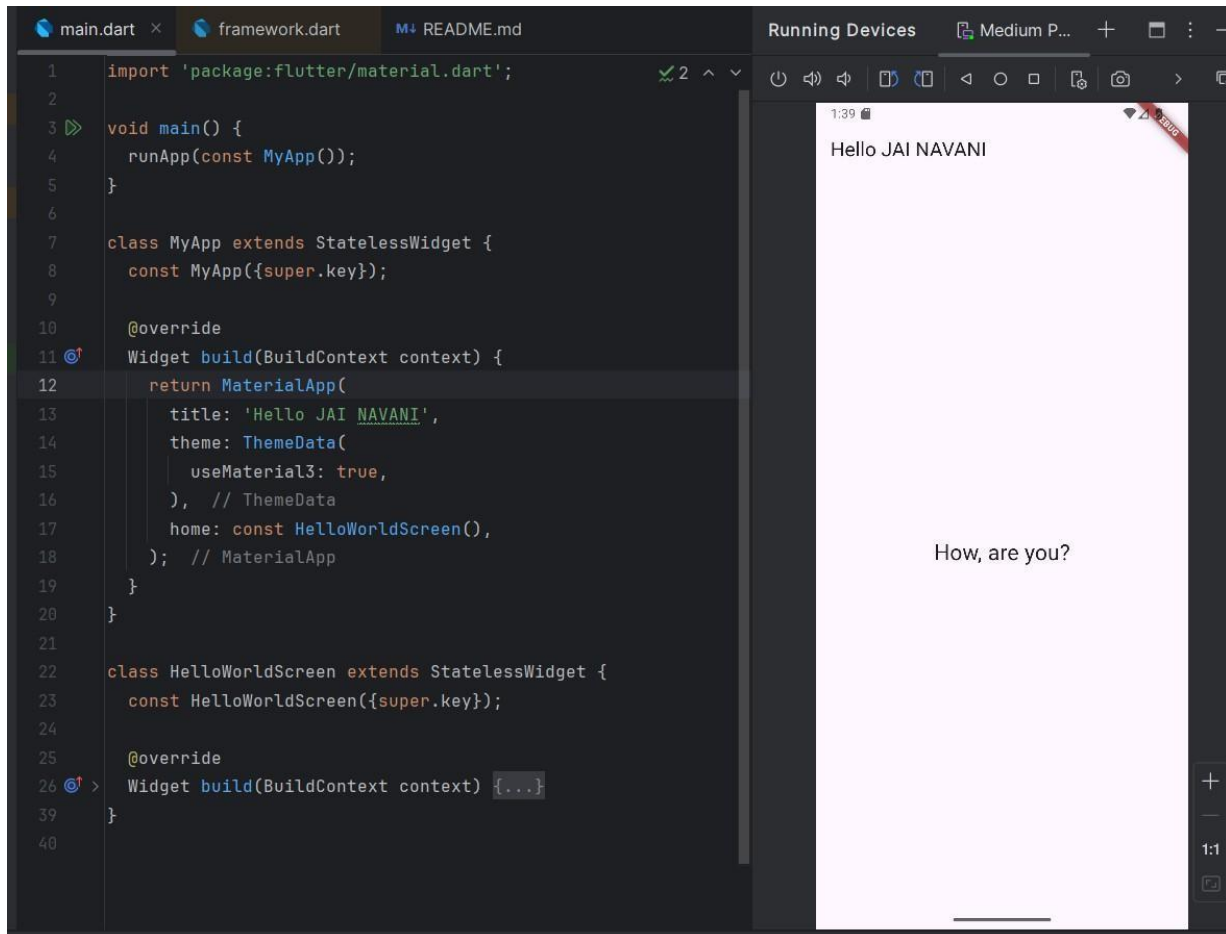
Step 8.1: To set an Android emulator, go to Android Studio > Tools > Android > AVD Manager and select Create Virtual Device. Or, go to Help->Find Action->Type Emulator in the search box. You will get the following screen.



Step 8.2: Choose your device definition and click on Next.

Step 8.3: Select the system image for the latest Android version and click on Next.

Step 8.4: Now, verify the all AVD configuration. If it is correct, click on Finish. The following screen appears.



Step 8.5: Last, click on the icon pointed into the red color rectangle. The Android emulator displayed as below screen.

Step 9: Now, install Flutter and Dart plugin for building Flutter application in Android Studio. These plugins provide a template to create a Flutter application, give an option to run and debug Flutter application in the Android Studio itself. Do the following steps to install these plugins.

Step 9.1: Open the Android Studio and then go to File->Settings->Plugins

Step 9.2: Restart the Android Studio.

MAD & PWA Lab

Journal

Experiment No.	02
Experiment Title.	To design Flutter UI by including common widgets.
Roll No.	30
Name	Jai Navani
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO2: Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation
Grade:	

MPL EXPERIMENT : 2

D15A-30

Jai navani

Login_page UI:

Theory:

The page has a minimalistic and clean design, focusing on user experience.

Uses consistent theming with teal and white colors.

Easy navigation is implemented using `Navigator.pushNamed()`.

The shopping cart icon and "Buy or Sell" text immediately communicate the app's purpose.

The full-width login button ensures accessibility and ease of use.

Widgets Used in the Login Page:

Widget & properties

Scaffold

The Scaffold widget provides the basic layout structure of the screen, including background color, body, and overall design.

Padding

The Padding widget is used to ensure that the content inside the screen doesn't stick to the edges and maintains a visually appealing layout.

`padding: EdgeInsets.symmetric(horizontal: 24.0);` This ensures uniform horizontal spacing on both sides.

Column

The Column widget is used to arrange UI elements vertically.

It has properties:

`mainAxisAlignment: MainAxisAlignment.center` → Centers all child widgets in the middle of the screen.

`crossAxisAlignment: CrossAxisAlignment.center` → Aligns the content to the center horizontally.
Icon

`Icon(Icons.shopping_cart, size: 100.0, color: Colors.white)`

This displays a shopping cart icon at the top of the page, representing an e-commerce or marketplace app.

SizedBox

Used to create spacing between widgets.

SizedBox(height: 20) → Adds space between the icon and text.
SizedBox(height: 40) → Adds space between the text and the button.
Text

Text('Buy or Sell') → Displays a heading in bold with a larger font.

TextStyle properties:

color: Colors.white → Makes the text color white.
fontSize: 32.0 → Sets the font size to 32 for emphasis.
fontWeight: FontWeight.bold → Makes the text bold.

ElevatedButton

This is the primary login button that allows users to navigate to the Sign-In page.
onPressed: Defines what happens when the button is clicked (it navigates to the SignInPage).
Navigator.pushNamed(context, '/signin', arguments: 'abc@gmail.com'); → Sends the user to the Sign-In page while passing an email argument.
Button Styling (using ElevatedButton.styleFrom):
backgroundColor: Colors.white → Button background is white.
foregroundColor: Colors.teal → Button text is teal.
minimumSize: Size(double.infinity, 50) → Button stretches across the screen with a height of 50px.
shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(8.0)) → Rounds the button corners for a modern look.

CODE:

```
import 'package:flutter/material.dart';

class LoginPage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: Colors.teal, // Green background
      body: Padding(
        padding: const EdgeInsets.symmetric(horizontal: 24.0),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          crossAxisAlignment: CrossAxisAlignment.center,
          children: [
            Icon(
              Icons.shopping_cart,
              size: 100.0,
              color: Colors.white,
            ),
          ],
        ),
      ),
    );
  }
}
```

```

    SizedBox(height: 20),
    Text(
      'Buy or Sell',
      style: TextStyle(
        color: Colors.white,
        fontSize: 32.0,
        fontWeight: FontWeight.bold,
      ),
    ),
    SizedBox(height: 40),
    ElevatedButton(
      onPressed: () {
        // Navigate to SignInPage with the email argument
        Navigator.pushNamed(
          context,
          '/signin',
          arguments: 'abc@gmail.com',
        );
      },
      style: ElevatedButton.styleFrom(
        backgroundColor: Colors.white,
        foregroundColor: Colors.teal,
        minimumSize: Size(double.infinity, 50), // Full width button
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(8.0),
        ),
      ),
      child: Text(
        "Login with Email",
        style: TextStyle(fontSize: 16.0, fontWeight: FontWeight.bold),
      ),
    ),
  ],
),
);
}
}

```

4:32



Buy or Sell

Login with Email

Signin_page UI:

Theory:

1. Scaffold

- The Scaffold widget is the root structure of the page, providing the AppBar and body content.

2. AppBar

- Title: Displays "Sign In", helping users understand the current screen.
- Background Color: Colors.teal, maintaining a consistent theme.
- Leading Button:
 - `IconButton(icon: Icon(Icons.arrow_back), onPressed: () { Navigator.pop(context); })`
 - Allows users to go back to the previous page (LoginPage).

3. Padding

- The entire page content is wrapped in a Padding widget with `EdgeInsets.all(16.0)`, ensuring proper spacing around elements.

4. Column

- Used for vertical alignment of elements (email field, password field, and button).
- `mainAxisAlignment: MainAxisAlignment.center` → Ensures content stays centered on the page.

5. TextField (Email Input Field)

- `controller: emailController` → Captures user input for email.
- `labelText: 'Email'` → Adds a label inside the input box.
- Styling:
 - `labelStyle: TextStyle(color: Colors.teal)`: The label is in teal.
 - `border: OutlineInputBorder()`: Adds a border around the input box.
 - `enabledBorder: OutlineInputBorder(borderSide: BorderSide(color: Colors.teal))`: Ensures a teal border when inactive.
 - `focusedBorder: OutlineInputBorder(borderSide: BorderSide(color: Colors.teal, width: 2.0))`: Thicker teal border when focused.

6. SizedBox (Spacing)

- `SizedBox(height: 20)`: Creates spacing between elements (email, password, and button).

7. TextField (Password Input Field)

- controller: passwordController → Captures user input for password.
- obscureText: true → Hides the password for security.
- Styled similarly to the email input field.

8. ElevatedButton (Sign-In Button)

- onPressed: Defines the authentication logic.
 - If credentials are correct (abc@gmail.com and 123), the user is redirected to the HomePage (Navigator.pushReplacementNamed(context, '/home');).
 - If credentials are incorrect, an error message appears using ScaffoldMessenger.of(context).showSnackBar().
- Button Styling (ElevatedButton.styleFrom):
 - backgroundColor: Colors.teal: Button color is teal.
 - foregroundColor: Colors.white: Text color is white.
 - minimumSize: Size(double.infinity, 50): Full-width button for better usability.
 - borderRadius: BorderRadius.circular(8.0): Rounded corners for a modern look.

Code:

```
import 'package:flutter/material.dart';

class SignInPage extends StatelessWidget {
  final TextEditingController emailController = TextEditingController();
  final TextEditingController passwordController = TextEditingController();

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("Sign In"),
        backgroundColor: Colors.teal, // Green theme
        leading: IconButton(
          icon: Icon(Icons.arrow_back),
          onPressed: () {
            Navigator.pop(context); // Go back to LoginPage
          },
        ),
      ),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
```

[illegible]

```
        content: Text('Invalid email or password'),
        backgroundColor: Colors.red,
      ),
    );
  }
},
style: ElevatedButton.styleFrom(
  backgroundColor: Colors.teal,
  foregroundColor: Colors.white,
  minimumSize: Size(double.infinity, 50), // Full width button
  shape: RoundedRectangleBorder(
    borderRadius: BorderRadius.circular(8.0),
  ),
),
child: Text(
  "Sign In",
  style: TextStyle(fontSize: 16.0, fontWeight: FontWeight.bold),
),
),
],
),
),
);
}
```

4:42



DEBUG



Sign In



Email

abc@gmail.com

Password

...

Sign In

MAD & PWA Lab

Journal

Experiment No.	03
Experiment Title.	To include icons, images, fonts in Flutter app
Roll No.	30
Name	Jai Navani
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO2: Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation
Grade:	

MPL EXPERIMENT : 3
D15A-30
Jai navani
(A page with basic ui and images)

Home_page UI:

Theory:

The HomePage is a stateful widget (StatefulWidget) that manages the bottom navigation and switches between different pages like Home, Chat, Sell, My Ads, and Account.

Key Components:

- `_currentIndex`: Tracks which tab is currently selected.
- `_onItemTapped(int index)`: Updates `_currentIndex` to switch tabs.
- `_pages` List: Contains all five pages, and the selected page is displayed using `_currentIndex`.
- `BottomNavigationBar`: Provides a tab bar at the bottom for easy navigation.

Widgets & Properties (Home Page)

- `StatefulWidget (HomePage)`: Since navigation changes dynamically, HomePage is a `StatefulWidget`. `_HomePageState` manages the selected tab and updates the UI when a tab is clicked.
- `Scaffold`: Provides the main structure of the app, including `AppBar`, `body`, and `BottomNavigationBar`.
- `AppBar`: `title: Text([...]_currentIndex)` → Displays the current tab title dynamically. `backgroundColor: Colors.teal` → Matches the app theme.
- `BottomNavigationBar`: Allows navigation between Home, Chat, Sell, My Ads, and Account. `currentIndex` tracks the selected tab.

onTap: _onItemTapped updates _currentIndex, reloading the UI.

selectedItemColor: Colors.teal highlights the active tab.

- HomeContentPage (Main Home Screen)

Includes Search Bar, Categories Grid, and Recommendations List.

- Search Bar (TextField):

hintText: "Search 'Mobiles'" → Placeholder text for searching.

prefixIcon: Icon(Icons.search, color: Colors.teal) → Search icon inside the field.

border: OutlineInputBorder(borderRadius: BorderRadius.circular(30.0)) → Stylish rounded edges.

- Category Grid (GridView.count)

Displays categories like Cars, Properties, Mobiles, etc. in a 4-column grid.

GestureDetector detects taps and updates selectedCategory.

Uses CategoryItem to show icons with labels.

- CategoryItem Widget

CircleAvatar(radius: 24.0, backgroundColor: Colors.teal[100]) → Creates a circular button with an icon.

Icon(icon, color: Colors.teal, size: 30) → Ensures icon visibility.

Text(label, style: TextStyle(fontSize: 12.0, color: Colors.teal)) → Displays category name below the icon.

- Recommendations List (ListView.builder)

Dynamically displays filtered product recommendations based on selectedCategory.

ListView.builder creates cards for each product.

Uses RecommendationItem to structure product details.

- RecommendationItem Widget

Displays product image, title, price, and location inside a Card.

leading: Image.asset(imageUrl, width: 50, height: 50, fit: BoxFit.cover) → Loads the product image.

- title: Text(title), subtitle: Text('\$price\n\$location') → Shows product details.
- trailing: Icon(Icons.favorite_border, color: Colors.teal) → Favorite button for liking items.

Code:

```
import 'package:flutter/material.dart';
import 'chat_page.dart';
import 'sell_page.dart';
import 'my_ads_page.dart';
import 'account_page.dart';

class HomePage extends StatefulWidget {
  @override
  _HomePageState createState() => _HomePageState();
}

class _HomePageState extends State<HomePage> {
  int _currentIndex = 0;

  final List<Widget> _pages = [
    HomeContentPage(), // Main Home Page
    ChatPage(),        // Chat Page
    SellPage(),         // Sell Page
    MyAdsPage(),        // My Ads Page
    AccountPage(),      // Account Page
  ];

  void _onItemTapped(int index) {
    setState(() {
      _currentIndex = index;
    });
  }
}
```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text([
        'Home', 'Chat', 'Sell', 'My Ads', 'Account'
      ][_currentIndex]),
      backgroundColor: Colors.teal,
    ),
    body: _pages[_currentIndex],
    bottomNavigationBar: BottomNavigationBar(
      currentIndex: _currentIndex,
      onTap: _onItemTapped,
      selectedItemColor: Colors.teal,
      unselectedItemColor: Colors.grey,
      items: [
        BottomNavigationBarItem(icon: Icon(Icons.home), label: 'Home'),
        BottomNavigationBarItem(icon: Icon(Icons.chat), label: 'Chat'),
        BottomNavigationBarItem(icon: Icon(Icons.add_circle, size: 36.0), label: 'Sell'),
        BottomNavigationBarItem(icon: Icon(Icons.post_add), label: 'My Ads'),
        BottomNavigationBarItem(icon: Icon(Icons.account_circle), label: 'Account'),
      ],
    ),
  );
}

```

```

class HomeContentPage extends StatefulWidget {
  @override
  _HomeContentPageState createState() => _HomeContentPageState();
}

```

```

class _HomeContentPageState extends State<HomeContentPage> {
  String selectedCategory = "All";
  final Map<String, List<String>> categoryItems = {
    "All": ["Car 1", "Car 2", "Car 3", "Car 4", "Car 5"],
    "Cars": ["Car 1", "Car 2", "Car 3"],
    "Properties": ["House 1", "House 2"],
    "Mobiles": ["Phone 1", "Phone 2"],
    "Jobs": ["Job 1", "Job 2"],
    "Bikes": ["Bike 1", "Bike 2"],
    "Electronics": ["TV 1", "Laptop 2"],
    "Commercial": ["Truck 1"],
  }
}

```

```
"Furniture": ["Sofa 1"],  
};
```

```
final Map<String, IconData> categoryIcons = {  
  "Cars": Icons.directions_car,  
  "Properties": Icons.home,  
  "Mobiles": Icons.smartphone,  
  "Jobs": Icons.work,  
  "Bikes": Icons.motorcycle,  
  "Electronics": Icons.tv,  
  "Commercial": Icons.local_shipping,  
  "Furniture": Icons.chair,  
};
```

```
@override  
Widget build(BuildContext context) {  
  return SingleChildScrollView(  
    child: Padding(  
      padding: const EdgeInsets.all(8.0),  
      child: Column(  
        crossAxisAlignment: CrossAxisAlignment.start,  
        children: [  
          // Search Bar  
          Container(  
            margin: EdgeInsets.symmetric(vertical: 8.0),  
            child: TextField(  
              decoration: InputDecoration(  
                hintText: 'Search "Mobiles"',  
                prefixIcon: Icon(Icons.search, color: Colors.teal),  
                filled: true,  
                fillColor: Colors.grey[200],  
                border: OutlineInputBorder(  
                  borderRadius: BorderRadius.circular(30.0),  
                  borderSide: BorderSide.none,  
                ),  
              ),  
            ),  
          ),  
          SizedBox(height: 16.0),  
          // Categories  
          GridView.count(  
            crossAxisCount: 4,  
            shrinkWrap: true,  
            physics: NeverScrollableScrollPhysics(),
```

```

children: categoryItems.keys.map((category) {
  return GestureDetector(
    onTap: () {
      setState(() {
        selectedCategory = category;
      });
    },
    child: CategoryItem(
      icon: categoryIcons[category] ?? Icons.category, // Default icon if not found
      label: category,
    ),
  );
}).toList(),
),
SizedBox(height: 16.0),
// Fresh Recommendations Header
Text(
  'Fresh Recommendations',
  style: TextStyle(fontSize: 18.0, fontWeight: FontWeight.bold, color: Colors.teal),
),
SizedBox(height: 8.0),
// Recommendations List
ListView.builder(
  itemCount: categoryItems[selectedCategory]!.length,
  shrinkWrap: true,
  physics: NeverScrollableScrollPhysics(),
  itemBuilder: (context, index) {
    return RecommendationItem(
      imageUrl: 'assets/images/car.jpg', // Local image
      title: categoryItems[selectedCategory]![index],
      price: '₹${(index + 1) * 10000}',
      location: 'Location ${index + 1}',
    );
  },
),
],
),
);
}
}

```

```

// Category Item Widget
class CategoryItem extends StatelessWidget {

```

```
final IconData icon;  
final String label;
```

```
CategoryItem({required this.icon, required this.label});
```

```
@override
```

```
Widget build(BuildContext context) {  
  return Column(  
    mainAxisAlignment: MainAxisAlignment.min,  
    children: [  
      CircleAvatar(  
        radius: 24.0,  
        backgroundColor: Colors.teal[100],  
        child: Icon(icon, color: Colors.teal, size: 30), // Ensuring icon visibility  
      ),  
      SizedBox(height: 4.0),  
      Text(label, style: TextStyle(fontSize: 12.0, color: Colors.teal)),  
    ],  
  );  
}
```

```
// Recommendation Item Widget
```

```
class RecommendationItem extends StatelessWidget {  
  final String imageUrl;  
  final String title;  
  final String price;  
  final String location;
```

```
  RecommendationItem({  
    required this.imageUrl,  
    required this.title,  
    required this.price,  
    required this.location,  
  });
```

```
@override
```

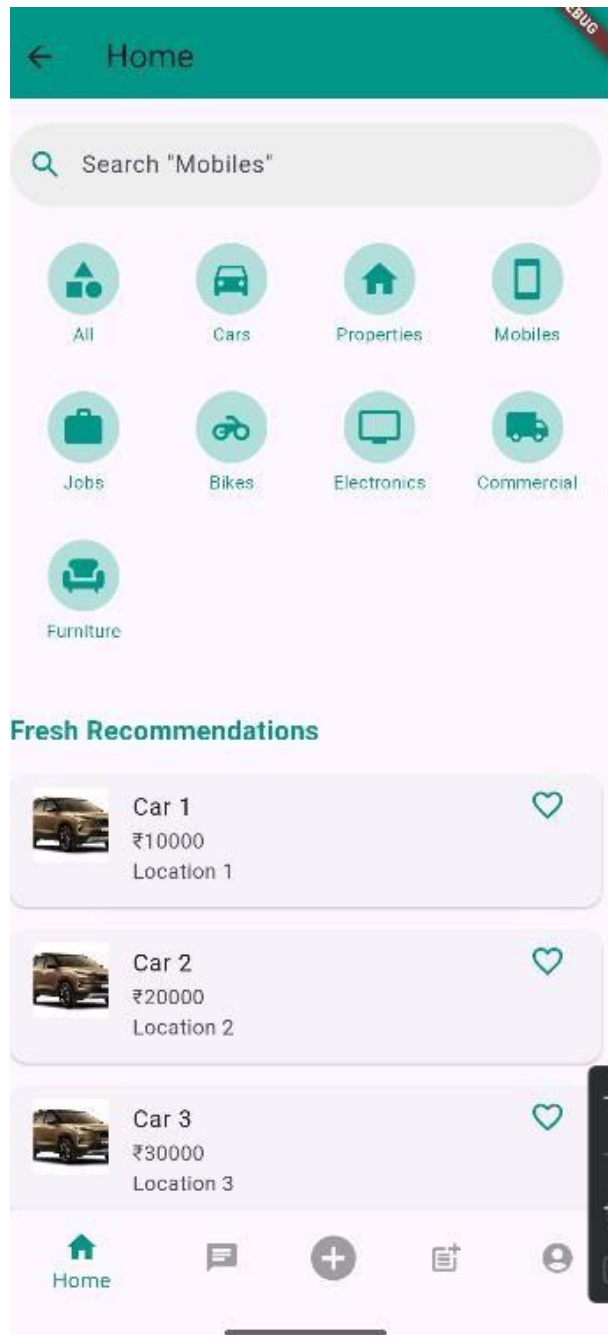
```
Widget build(BuildContext context) {  
  return Card(  
    margin: EdgeInsets.symmetric(vertical: 8.0),  
    child: ListTile(  
      leading: Image.asset(imageUrl, width: 50, height: 50, fit: BoxFit.cover),  
      title: Text(title),  
      subtitle: Text('$price\n$location'),
```

```

        isThreeLine: true,
        trailing: Icon(Icons.favorite_border, color: Colors.teal),
      ),
    );
  }
}

```

OUTPUT:



MAD & PWA Lab

Journal

Experiment No.	04
Experiment Title.	To create an interactive Form using form widget
Roll No.	30
Name	Jai Navani
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO2: Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation
Grade:	

MPL EXPERIMENT : 4
D15A-30
Jai navani
(an interactive page made named chat page)

Chat_page UI:

Theory:

The ChatPage is a simple yet effective chat interface that allows users to send and display messages dynamically. It maintains a list of messages and updates the UI in real time when a new message is sent. Below is a breakdown of the widgets used and their functionality.

Widget & properties:

1. StatefulWidget (ChatPage)

- Since chat messages need to be updated dynamically, we use a StatefulWidget.
- The _ChatPageState class manages the state, including the list of messages and the text input controller.

2. Scaffold

- The Scaffold widget provides the basic structure, including an AppBar and the main body content.

3. AppBar

- title: Text("Chat") → Displays the title "Chat" at the top.
- backgroundColor: Colors.teal → Matches the theme with a teal-colored AppBar.

4. Column

Used to organize two main sections:

1. Expanded (List of Messages)
2. Message Input & Send Button

5. Expanded (ListView.builder for Chat Messages)

- The Expanded widget ensures the chat messages take up all available space above the input field.
- ListView.builder dynamically generates message bubbles from _messages list.

- `reverse: true` → Shows the latest messages at the bottom (like most chat apps).

Message Bubble Design

Each message is styled based on whether it is a user message or received message:

- User message (right-aligned, light color) → `Colors.teal.withOpacity(0.2)`
- Received message (left-aligned, dark teal) → `Colors.teal`
- Text color:
 - User messages: `black`
 - Received messages: `white`
- Padding & Border Radius:
 - `margin: EdgeInsets.symmetric(vertical: 5.0, horizontal: 10.0)`
 - `padding: EdgeInsets.symmetric(vertical: 10.0, horizontal: 14.0)`
 - `borderRadius: BorderRadius.circular(8.0)`

6. Message Input Field & Send Button

A Row is used to align the input field and the send button horizontally.

TextField (Message Input)

- `controller: _messageController` → Captures user input.
- `hintText: "Type a message"` → Placeholder text inside the input field.
- `border: OutlineInputBorder(borderRadius: BorderRadius.circular(8.0))` → Stylish rounded input box.

ElevatedButton (Send Button)

- Logic (`onPressed`):
 - Retrieves text from `_messageController`.
 - If not empty, inserts the message at index 0 (so latest messages appear at the bottom).
 - Clears the input field after sending.
- Button Styling (`ElevatedButton.styleFrom`):
 - `backgroundColor: Colors.teal` → Matches the theme.
 - `padding: EdgeInsets.all(12.0)` → Proper button sizing.
 - `child: Icon(Icons.send, color: Colors.white)` → White send icon.

Code:

```
import 'package:flutter/material.dart';

class ChatPage extends StatefulWidget {
  @override
  _ChatPageState createState() => _ChatPageState();
}

class _ChatPageState extends State<ChatPage> {
  final TextEditingController _messageController = TextEditingController();
  final List<String> _messages = []; // List to store chat messages

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("Chat"),
        backgroundColor: Colors.teal,
      ),
      body: Column(
        children: [
          // Chat messages list
          Expanded(
            child: ListView.builder(
              itemCount: _messages.length,
              reverse: true, // Show the latest message at the bottom
              itemBuilder: (context, index) {
                final message = _messages[index];
                final isUserMessage = index % 2 == 0; // Simulate sender/receiver

                return Align(
                  alignment: isUserMessage
                    ? Alignment.centerRight
                    : Alignment.centerLeft,
                  child: Container(
                    margin: EdgeInsets.symmetric(
                      vertical: 5.0, horizontal: 10.0),
                    padding: EdgeInsets.symmetric(
                      vertical: 10.0, horizontal: 14.0),
                    decoration: BoxDecoration(
                      color: isUserMessage
                        ? Colors.teal.withOpacity(0.2)
```

```

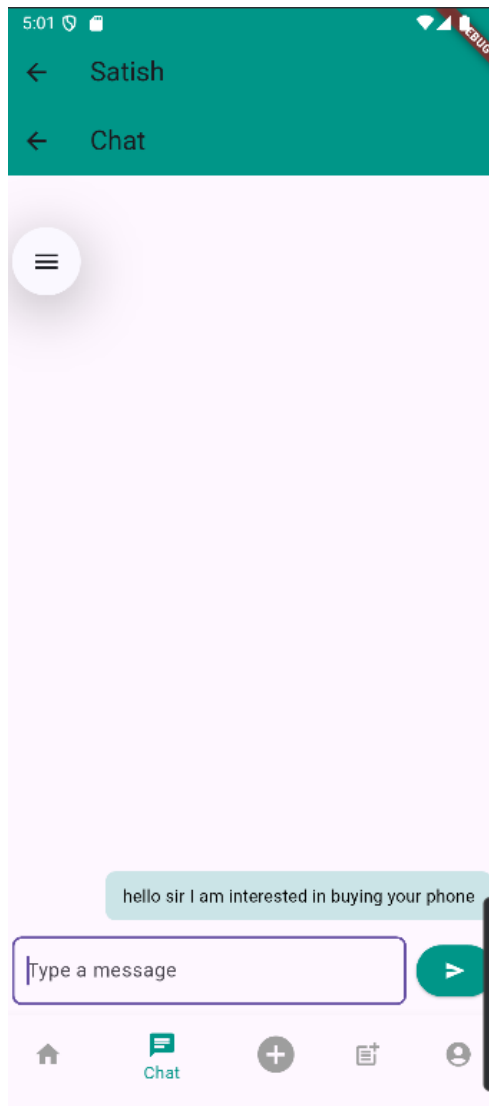
        : Colors.teal,
        borderRadius: BorderRadius.circular(8.0),
      ),
      child: Text(
        message,
        style: TextStyle(
          color: isUserMessage ? Colors.black : Colors.white,
        ),
      ),
    ),
  );
},
),
),
);
// Input field and send button
Padding(
  padding: const EdgeInsets.all(8.0),
  child: Row(
    children: [
      Expanded(
        child: TextField(
          controller: _messageController,
          decoration: InputDecoration(
            hintText: "Type a message",
            border: OutlineInputBorder(
              borderRadius: BorderRadius.circular(8.0),
            ),
          ),
        ),
      ),
      ElevatedButton(
        onPressed: () {
          final message = _messageController.text;
          if (message.isNotEmpty) {
            setState(() {
              _messages.insert(0, message); // Add new message
            });
            _messageController.clear(); // Clear input field
          }
        },
        style: ElevatedButton.styleFrom(
          backgroundColor: Colors.teal,

```

```

padding: EdgeInsets.all(12.0),
),
child: Icon(Icons.send, color: Colors.white),
),
],
),
),
],
),
);
}
}

```



MAD & PWA Lab

Journal

Experiment No.	05
Experiment Title.	To apply navigation, routing and gestures in Flutter App
Roll No.	30
Name	Jai Navani
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO2: Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation
Grade:	

MPL EXPERIMENT : 5
D15A-30
Jai navani
(A page with navigation and routing)

Home_page UI:

Theory:

The HomePage is a stateful widget (StatefulWidget) that manages the bottom navigation and switches between different pages like Home, Chat, Sell, My Ads, and Account.

Key Components:

- `_currentIndex`: Tracks which tab is currently selected.
- `_onItemTapped(int index)`: Updates `_currentIndex` to switch tabs.
- `_pages` List: Contains all five pages, and the selected page is displayed using `_currentIndex`.
- `BottomNavigationBar`: Provides a tab bar at the bottom for easy navigation.

Widgets & Properties (Home Page)

- `StatefulWidget (HomePage)`: Since navigation changes dynamically, HomePage is a `StatefulWidget`. `_HomePageState` manages the selected tab and updates the UI when a tab is clicked.
- `Scaffold`: Provides the main structure of the app, including `AppBar`, `body`, and `BottomNavigationBar`.
- `AppBar`: `title: Text([...][_currentIndex])` → Displays the current tab title dynamically. `backgroundColor: Colors.teal` → Matches the app theme.
- `BottomNavigationBar`: Allows navigation between Home, Chat, Sell, My Ads, and Account. `currentIndex` tracks the selected tab.

onTap: _onItemTapped updates _currentIndex, reloading the UI.

selectedItemColor: Colors.teal highlights the active tab.

- HomeContentPage (Main Home Screen)

Includes Search Bar, Categories Grid, and Recommendations List.

- Search Bar (TextField):

hintText: "Search 'Mobiles'" → Placeholder text for searching.

prefixIcon: Icon(Icons.search, color: Colors.teal) → Search icon inside the field.

border: OutlineInputBorder(borderRadius: BorderRadius.circular(30.0)) → Stylish rounded edges.

- Category Grid (GridView.count)

Displays categories like Cars, Properties, Mobiles, etc. in a 4-column grid.

GestureDetector detects taps and updates selectedCategory.

Uses CategoryItem to show icons with labels.

- CategoryItem Widget

CircleAvatar(radius: 24.0, backgroundColor: Colors.teal[100]) → Creates a circular button with an icon.

Icon(icon, color: Colors.teal, size: 30) → Ensures icon visibility.

Text(label, style: TextStyle(fontSize: 12.0, color: Colors.teal)) → Displays category name below the icon.

- Recommendations List (ListView.builder)

Dynamically displays filtered product recommendations based on selectedCategory.

ListView.builder creates cards for each product.

Uses RecommendationItem to structure product details.

- RecommendationItem Widget

Displays product image, title, price, and location inside a Card.

leading: Image.asset(imageUrl, width: 50, height: 50, fit: BoxFit.cover) → Loads the product image.

- title: Text(title), subtitle: Text('\$price\n\$location') → Shows product details.
- trailing: Icon(Icons.favorite_border, color: Colors.teal) → Favorite button for liking items.

Code:

```
import 'package:flutter/material.dart';
import 'chat_page.dart';
import 'sell_page.dart';
import 'my_ads_page.dart';
import 'account_page.dart';

class HomePage extends StatefulWidget {
  @override
  _HomePageState createState() => _HomePageState();
}

class _HomePageState extends State<HomePage> {
  int _currentIndex = 0;

  final List<Widget> _pages = [
    HomeContentPage(), // Main Home Page
    ChatPage(),        // Chat Page
    SellPage(),         // Sell Page
    MyAdsPage(),        // My Ads Page
    AccountPage(),      // Account Page
  ];

  void _onItemTapped(int index) {
    setState(() {
      _currentIndex = index;
    });
  }
}
```



```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text([
        'Home', 'Chat', 'Sell', 'My Ads', 'Account'
      ][_currentIndex]),
      backgroundColor: Colors.teal,
    ),
    body: _pages[_currentIndex],
    bottomNavigationBar: BottomNavigationBar(
      currentIndex: _currentIndex,
      onTap: _onItemTapped,
      selectedItemColor: Colors.teal,
      unselectedItemColor: Colors.grey,
      items: [
        BottomNavigationBarItem(icon: Icon(Icons.home), label: 'Home'),
        BottomNavigationBarItem(icon: Icon(Icons.chat), label: 'Chat'),
        BottomNavigationBarItem(icon: Icon(Icons.add_circle, size: 36.0), label: 'Sell'),
        BottomNavigationBarItem(icon: Icon(Icons.post_add), label: 'My Ads'),
        BottomNavigationBarItem(icon: Icon(Icons.account_circle), label: 'Account'),
      ],
    ),
  );
}

```

```

class HomeContentPage extends StatefulWidget {
  @override
  _HomeContentPageState createState() => _HomeContentPageState();
}

```

```

class _HomeContentPageState extends State<HomeContentPage> {
  String selectedCategory = "All";
  final Map<String, List<String>> categoryItems = {
    "All": ["Car 1", "Car 2", "Car 3", "Car 4", "Car 5"],
    "Cars": ["Car 1", "Car 2", "Car 3"],
    "Properties": ["House 1", "House 2"],
    "Mobiles": ["Phone 1", "Phone 2"],
    "Jobs": ["Job 1", "Job 2"],
    "Bikes": ["Bike 1", "Bike 2"],
    "Electronics": ["TV 1", "Laptop 2"],
    "Commercial": ["Truck 1"],
  }
}

```

```
"Furniture": ["Sofa 1"],  
};
```

```
final Map<String, IconData> categoryIcons = {  
  "Cars": Icons.directions_car,  
  "Properties": Icons.home,  
  "Mobiles": Icons.smartphone,  
  "Jobs": Icons.work,  
  "Bikes": Icons.motorcycle,  
  "Electronics": Icons.tv,  
  "Commercial": Icons.local_shipping,  
  "Furniture": Icons.chair,  
};
```

```
@override  
Widget build(BuildContext context) {  
  return SingleChildScrollView(  
    child: Padding(  
      padding: const EdgeInsets.all(8.0),  
      child: Column(  
        crossAxisAlignment: CrossAxisAlignment.start,  
        children: [  
          // Search Bar  
          Container(  
            margin: EdgeInsets.symmetric(vertical: 8.0),  
            child: TextField(  
              decoration: InputDecoration(  
                hintText: 'Search "Mobiles"',  
                prefixIcon: Icon(Icons.search, color: Colors.teal),  
                filled: true,  
                fillColor: Colors.grey[200],  
                border: OutlineInputBorder(  
                  borderRadius: BorderRadius.circular(30.0),  
                  borderSide: BorderSide.none,  
                ),  
              ),  
            ),  
          ),  
          SizedBox(height: 16.0),  
          // Categories  
          GridView.count(  
            crossAxisCount: 4,  
            shrinkWrap: true,  
            physics: NeverScrollableScrollPhysics(),
```

```

children: categoryItems.keys.map((category) {
  return GestureDetector(
    onTap: () {
      setState(() {
        selectedCategory = category;
      });
    },
    child: CategoryItem(
      icon: categoryIcons[category] ?? Icons.category, // Default icon if not found
      label: category,
    ),
  );
}).toList(),
),
 SizedBox(height: 16.0),
// Fresh Recommendations Header
Text(
  'Fresh Recommendations',
  style: TextStyle(fontSize: 18.0, fontWeight: FontWeight.bold, color: Colors.teal),
),
SizedBox(height: 8.0),
// Recommendations List
ListView.builder(
  itemCount: categoryItems[selectedCategory]!.length,
  shrinkWrap: true,
  physics: NeverScrollableScrollPhysics(),
  itemBuilder: (context, index) {
    return RecommendationItem(
      imageUrl: 'assets/images/car.jpg', // Local image
      title: categoryItems[selectedCategory]![index],
      price: '₹${(index + 1) * 10000}',
      location: 'Location ${index + 1}',
    );
  },
),
],
),
),
);
}
}

```

```

// Category Item Widget
class CategoryItem extends StatelessWidget {

```

```
final IconData icon;  
final String label;
```

```
CategoryItem({required this.icon, required this.label});
```

```
@override
```

```
Widget build(BuildContext context) {  
  return Column(  
    mainAxisAlignment: MainAxisAlignment.min,  
    children: [  
      CircleAvatar(  
        radius: 24.0,  
        backgroundColor: Colors.teal[100],  
        child: Icon(icon, color: Colors.teal, size: 30), // Ensuring icon visibility  
      ),  
      SizedBox(height: 4.0),  
      Text(label, style: TextStyle(fontSize: 12.0, color: Colors.teal)),  
    ],  
  );  
}
```

```
// Recommendation Item Widget
```

```
class RecommendationItem extends StatelessWidget {  
  final String imageUrl;  
  final String title;  
  final String price;  
  final String location;
```

```
  RecommendationItem({  
    required this.imageUrl,  
    required this.title,  
    required this.price,  
    required this.location,  
  });
```

```
@override
```

```
Widget build(BuildContext context) {  
  return Card(  
    margin: EdgeInsets.symmetric(vertical: 8.0),  
    child: ListTile(  
      leading: Image.asset(imageUrl, width: 50, height: 50, fit: BoxFit.cover),  
      title: Text(title),  
      subtitle: Text('$price\n$location'),
```

```

        isThreeLine: true,
        trailing: Icon(Icons.favorite_border, color: Colors.teal),
      ),
    );
  }
}

```

ROUTED PAGE:

```

import 'package:flutter/material.dart';

class SellPage extends StatefulWidget {
  @override
  _SellPageState createState() => _SellPageState();
}

class _SellPageState extends State<SellPage> {
  final _formKey = GlobalKey<FormState>();
  final _nameController = TextEditingController();
  final _priceController = TextEditingController();

  String? _selectedCategory;

  final List<String> _categories = [
    'Cars',
    'Properties',
    'Mobiles',
    'Jobs',
    'Bikes',
    'Electronics',
    'Commercial',
    'Furniture'
  ];

  void _submitForm() {
    if (_formKey.currentState!.validate() && _selectedCategory != null) {
      // Handle form submission logic
      showDialog(
        context: context,
        builder: (context) {
          return AlertDialog(
            title: Text('Product Listed'),
            content: Text(

```

```

        'Name: ${_nameController.text}\nCategory: $_selectedCategory\nPrice:
$_priceController.text}',
      ),
      actions: [
        TextButton(
          onPressed: () {
            Navigator.of(context).pop();
          },
          child: Text('Close'),
        ),
      ],
    );
  },
);
}
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text('Sell Product'),
      backgroundColor: Colors.teal,
    ),
    body: Padding(
      padding: const EdgeInsets.all(16.0),
      child: Form(
        key: _formKey,
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            // Name Input
            TextFormField(
              controller: _nameController,
              decoration: InputDecoration(
                labelText: 'Product Name',
                border: OutlineInputBorder(
                  borderRadius: BorderRadius.circular(10.0),
                ),
              ),
              validator: (value) {
                if (value == null || value.isEmpty) {
                  return 'Please enter the product name';
                }
              }
            )
          ],
        ),
      ),
    ),
  );
}

```

```

        return null;
      },
    ),
    SizedBox(height: 16.0),

    // Category Dropdown
    DropdownButtonFormField<String>(
      decoration: InputDecoration(
        labelText: 'Category',
        border: OutlineInputBorder(
          borderRadius: BorderRadius.circular(10.0),
        ),
      ),
    ),
    value: _selectedCategory,
    items: _categories.map((category) {
      return DropdownMenuItem(
        value: category,
        child: Text(category),
      );
    }).toList(),
    onChanged: (value) {
      setState(() {
        _selectedCategory = value;
      });
    },
    validator: (value) {
      if (value == null) {
        return 'Please select a category';
      }
      return null;
    },
  ),
  SizedBox(height: 16.0),

  // Price Input
  TextFormField(
    controller: _priceController,
    keyboardType: TextInputType.number,
    decoration: InputDecoration(
      labelText: 'Price',
      border: OutlineInputBorder(
        borderRadius: BorderRadius.circular(10.0),
      ),
    ),
  ),

```

```

        validator: (value) {
          if (value == null || value.isEmpty) {
            return 'Please enter the price';
          }
          if (double.tryParse(value) == null) {
            return 'Please enter a valid number';
          }
          return null;
        },
      ),
      SizedBox(height: 16.0),

      // Submit Button
      Center(
        child: ElevatedButton(
          onPressed: _submitForm,
          style: ElevatedButton.styleFrom(
            //primary: Colors.teal,
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(10.0),
            ),
          ),
        ),
        child: Text('List Product'),
      ),
    ],
  ),
),
);
}
}

```

OUTPUT:

🔍 Search "Mobiles"



All



Cars



Properties



Mobiles



Jobs



Bikes



Electronics



Commercial



Furniture

Fresh Recommendations



Car 1

₹10000

Location 1



Car 2

₹20000

Location 2



Car 3

₹30000

Location 3



Home



5:18

Sell

Sell Product

Product Name

phone

Category

Mobiles

Price

1500

List Product

+

Sell

500

Product Listed

Name: phone

Category: Mobiles

Price: 1500

Close

MAD & PWA Lab

Journal

Experiment No.	06
Experiment Title.	To Connect Flutter UI with fireBase database
Roll No.	30
Name	Jai Navani
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO3: Analyze and Build production ready Flutter App by incorporating backend services and deploying on Android / iOS
Grade:	

MPL EXPERIMENT : 6
D15A-30
Jai navani
(with firebase connection)

New_login and signin_page:

Theory:

The SigninPage is a stateful widget that allows users to log in using their email and password. It interacts with Firebase Authentication to validate user credentials and redirects users to the home page upon successful login.

The NewLoginPage is a stateful widget that allows users to register using their email and password. It interacts with Firebase Authentication to create a new account.

Widgets & Properties (Home Page)

StatefulWidget (SigninPage)

- Used because the UI changes dynamically based on user interactions (e.g., login success or failure).

TextEditingController (for Input Fields)

- `_emailController`: Captures user input for the email field.
- `_passwordController`: Captures the password input.
- These controllers help retrieve user input when needed.

FirebaseAuth (`_auth`)

- `FirebaseAuth.instance` → Used to handle user authentication.
- `_auth.signInWithEmailAndPassword(email, password)` → Validates credentials with Firebase.

Scaffold (Main UI Structure)

- `AppBar` → Displays the title "Sign In" with a teal-colored background.
- `body` → Contains a `Column` that aligns widgets vertically.

Column (Main Layout)

- TextField (Email Input)
 - controller: `_emailController` → Captures user email.
 - decoration: `InputDecoration(labelText: "Email", border: OutlineInputBorder())` → Provides a label and a border.
- TextField (Password Input)
 - controller: `_passwordController` → Captures password.
 - obscureText: `true` → Hides password characters for security.
- ElevatedButton (Login Button)
 - onPressed: `_loginUser` → Calls `_loginUser()` function.
 - child: `Text("Login")` → Displays button text.

`_loginUser()` (Handles Login)

- `signInWithEmailAndPassword(email, password)`
 - Tries to log in the user using Firebase.
- If login is successful → Displays success message and redirects to the home page (`Navigator.pushReplacementNamed(context, '/home')`).
- If login fails → Shows an error message using `SnackBar`.

NEWLOGIN_PAGE:

FirebaseAuth (`_auth`)

- `FirebaseAuth.instance` → Used to register new users.
- `_auth.createUserWithEmailAndPassword(email, password)` → Creates a new account.

Column (Main Layout)

- TextField (Email Input)
 - controller: `_emailController` → Captures email input.
 - decoration: `InputDecoration(labelText: "Email")` → Provides a label.
- TextField (Password Input)
 - controller: `_passwordController` → Captures password input.
 - obscureText: `true` → Hides password characters.
- ElevatedButton (Register Button)
 - onPressed: `_registerUser` → Calls `_registerUser()` function.
 - child: `Text("Register")` → Displays button text.

`_registerUser()` (Handles Registration)

- createUserWithEmailAndPassword(email, password)
 - Creates a new Firebase account.

Code:

SIGIN_PAGE:

```
import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';

class SigninPage extends StatefulWidget {
  @override
  _SigninPageState createState() => _SigninPageState();
}

class _SigninPageState extends State<SigninPage> {
  final TextEditingController _emailController = TextEditingController();
  final TextEditingController _passwordController = TextEditingController();
  final FirebaseAuth _auth = FirebaseAuth.instance;

  Future<void> _loginUser() async {
    try {
      UserCredential userCredential = await _auth.signInWithEmailAndPassword(
        email: _emailController.text.trim(),
        password: _passwordController.text.trim(),
      );

      if (userCredential.user != null) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text('Login successful! Redirecting...')),
        );

        // Delay before navigation (Fixes async navigation issue)
        Future.delayed(Duration(seconds: 1), () {
          Navigator.pushReplacementNamed(context, '/home');
        });
      }
    } on FirebaseAuthException catch (e) {
```

```

        String errorMessage = "Login failed: ${e.message}";
        ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(errorMessage)));
    }
}

@override
Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(title: Text("Sign In"), backgroundColor: Colors.teal),
        body: Padding(
            padding: EdgeInsets.all(16.0),
            child: Column(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
                    TextField(
                        controller: _emailController,
                        decoration: InputDecoration(labelText: "Email", border: OutlineInputBorder()),
                    ),
                    SizedBox(height: 10),
                    TextField(
                        controller: _passwordController,
                        decoration: InputDecoration(labelText: "Password", border: OutlineInputBorder()),
                        obscureText: true,
                    ),
                    SizedBox(height: 20),
                    ElevatedButton(
                        onPressed: _loginUser, // Now it logs in instead of signing up
                        child: Text("Login"),
                    ),
                ],
            ),
        ),
    );
}
}

```

NEWLOGIN_PAGE:

```

import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';

```

```

class NewLoginPage extends StatefulWidget {
  @override
  _NewLoginPageState createState() => _NewLoginPageState();
}

class _NewLoginPageState extends State<NewLoginPage> {
  final TextEditingController _emailController = TextEditingController();
  final TextEditingController _passwordController = TextEditingController();
  final FirebaseAuth _auth = FirebaseAuth.instance;

  Future<void> _registerUser() async {
    try {
      await _auth.createUserWithEmailAndPassword(
        email: _emailController.text.trim(),
        password: _passwordController.text.trim(),
      );
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('User registered successfully!')),
      );
    } catch (e) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Registration failed: $e')),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("New Login")),
      body: Padding(
        padding: EdgeInsets.all(16.0),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            TextField(
              controller: _emailController,
              decoration: InputDecoration(labelText: "Email"),
            ),
            TextField(
              controller: _passwordController,
              decoration: InputDecoration(labelText: "Password"),
              obscureText: true,
            ),
          ],
        ),
      ),
    );
  }
}

```



```

        SizedBox(height: 20),
        ElevatedButton(
          onPressed: _registerUser,
          child: Text("Register"),
        ),
      ],
    ),
  ),
);
}
}

```

ACCOUNT_PAGE:

```

import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'signin_page.dart'; // Importing SignInPage

class AccountPage extends StatelessWidget {
  final FirebaseAuth _auth = FirebaseAuth.instance;

  @override
  Widget build(BuildContext context) {
    // Get the currently logged-in user
    User? user = _auth.currentUser;

    return Scaffold(
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Center(
              child: CircleAvatar(
                radius: 50,
                backgroundColor: Colors.teal[100],
                child: Icon(Icons.person, size: 50, color: Colors.teal),
              ),
            ),
            SizedBox(height: 16),
            Text('Name: John Doe', style: TextStyle(fontSize: 18)),
            SizedBox(height: 8),
            Text(

```

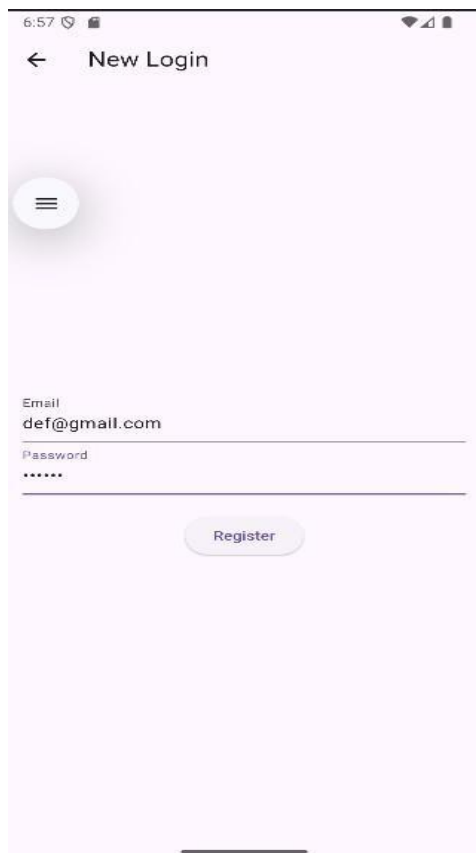
```

      'Email: ${user?.email ?? "Not Logged In"}', // Dynamically show email
      style: TextStyle(fontSize: 18),
    ),
    SizedBox(height: 8),
    Text('Phone: +91 9999999999', style: TextStyle(fontSize: 18)),
    SizedBox(height: 16),
    Divider(),
    ListTile(
      leading: Icon(Icons.settings, color: Colors.teal),
      title: Text('Settings'),
      onTap: () {},
    ),
    ListTile(
      leading: Icon(Icons.help, color: Colors.teal),
      title: Text('Help & Support'),
      onTap: () {},
    ),
    Spacer(),
    Center(
      child: ElevatedButton(
        onPressed: () async {
          await _auth.signOut(); // Logout user
          Navigator.pushReplacement(
            context,
            MaterialPageRoute(builder: (context) => SigninPage()),
          );
        },
        child: Text('Logout', style: TextStyle(fontSize: 18)),
        style: ElevatedButton.styleFrom(
          backgroundColor: Colors.red,
          padding: EdgeInsets.symmetric(horizontal: 32, vertical: 12),
        ),
      ),
    ),
  ],
),
),
);
}
}

```

OUTPUT:

Newlogin_page:

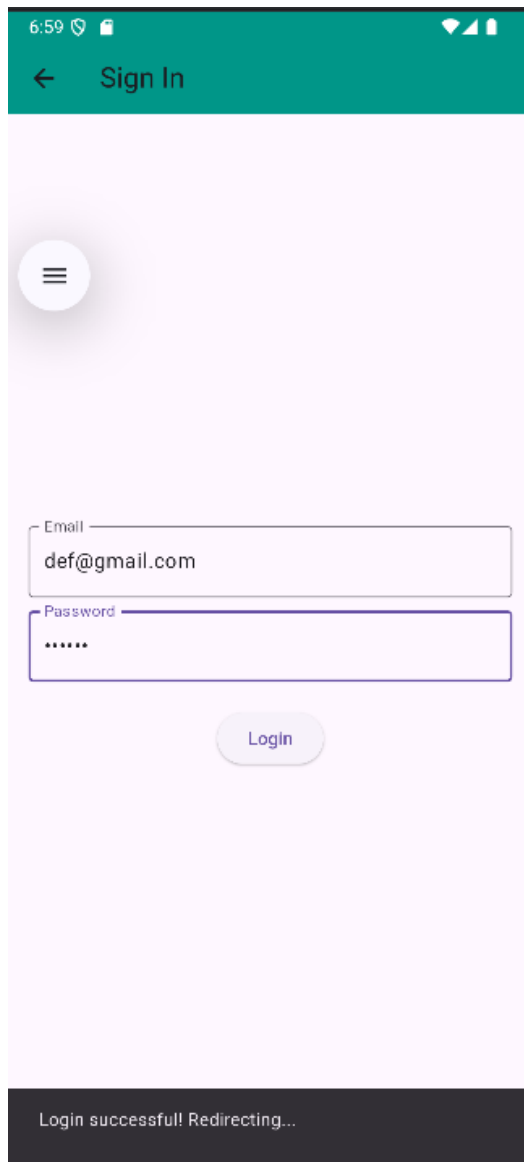


A mobile app screenshot of the 'New Login' page. The page has a light pink background. At the top, there's a status bar showing the time 6:57 and battery level. Below the status bar, there's a back arrow and the text 'New Login'. A hamburger menu icon is on the left. The main form has two input fields: 'Email' with the value 'def@gmail.com' and 'Password' with masked characters '*****'. Below the password field is a 'Register' button.

Search by email address, phone number, or user UID					Add user	↺	⋮
Identifier	Providers	Created ↓	Signed In	User UID			
def@gmail.com	✉	Mar 2, 2025	Mar 2, 2025	IMKSB0qlw4cyKXxeUweB2JP...			
abc@gmail.com	✉	Mar 2, 2025	Mar 2, 2025	DzDVpi5M7LclNQIS4uodONh...			
					Rows per page:	50 ▼	1 – 2 of 2 < >

User registered

Now when tried to logged in:



Email we tried was def@gmail.com

Email: def@gmail.com
Phone: +91 9999999999

 Settings

 Help & Support

Logout

Then tried with other email:

Email: abc@gmail.com
Phone: +91 9999999999

 Settings

 Help & Support

Logout

MAD & PWA Lab

Journal

Experiment No.	07
Experiment Title.	To write meta data of your Ecommerce PWA in a Web app manifest file to enable “add to homescreen feature”.
Roll No.	30
Name	Jai Navani
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO4: Understand various PWA frameworks and their requirements
Grade:	

PWA Experiment -7

Jai Navani

D15A 30

AIM: To write meta data of your Ecommerce PWA in a Web app manifest file to enable “add to homescreen feature”.

Theory:-

Regular Web Application

A regular web application is a website designed to be accessible on various devices, ensuring that content adjusts dynamically to different screen sizes. Built using web technologies such as HTML, CSS, JavaScript, and Ruby, these applications function through a browser. While they can leverage some native device features, their functionality is dependent on browser compatibility. For instance, a feature may work on Google Chrome but not on Safari or Mozilla Firefox due to browser limitations.

Progressive Web Application (PWA)

A Progressive Web Application (PWA) is an advanced version of a regular web app, integrating additional features to enhance the user experience. PWAs combine the best elements of desktop and mobile applications, delivering a seamless experience across platforms.

Key Differences Between PWAs and Regular Web Apps

1. Native-Like Experience

While both PWAs and regular web apps utilize standard web technologies like HTML, CSS, and JavaScript, PWAs offer a user experience similar to native mobile applications. PWAs can access device-specific features such as push notifications without relying on a particular browser, creating a smoother, more integrated experience.

2. Ease of Access

Unlike traditional mobile apps that require time-consuming downloads and storage space, PWAs can be installed directly via a URL. Users can add a PWA to their home screen with a simple link, eliminating installation complexities and ensuring easy access while keeping brand presence strong.

3. Enhanced Performance

PWAs utilize caching mechanisms to pre-load content, such as text, stylesheets, and images, allowing for faster loading times. This significantly improves user engagement and retention by reducing waiting periods, ultimately benefiting businesses by increasing interaction rates.

4. Improved User Engagement

PWAs efficiently leverage push notifications and native device features to keep users engaged. Unlike regular web apps, their functionality is not restricted by browser dependencies, enabling businesses to notify users about updates, offers, and promotions without disruptions.

5. Real-Time Updates

A major advantage of PWAs is their ability to update automatically without requiring users to download new versions from an app store. Developers can push updates directly from the server, ensuring that users always access the latest features and improvements instantly.

6. Search Engine Optimization (SEO) Benefits

Since PWAs function within web browsers, they can be indexed by search engines, improving their visibility in search results. This gives them a strategic advantage over native apps, which are limited to app store searches.

7. Cost-Effective Development Unlike native mobile apps, PWAs do not require approval or submission to app stores, reducing development and maintenance costs.

Advantages and Limitations of PWAs

Advantages:

- **Progressive:** Compatible with all browsers and devices, following the principle of progressive enhancement.
- **Responsive:** Adapts to different screen sizes, including desktops, tablets, and smartphones.
- **App-Like Feel:** Mimics the experience of native applications in navigation and interaction.
- **Always Updated:** Service workers ensure real-time updates without requiring user intervention.
- **Secure:** Delivered over HTTPS, ensuring secure data transfer and protection against cyber threats.
- **SEO-Friendly:** Can be indexed by search engines, enhancing discoverability.
- **Re-Engagement:** Enables push notifications to encourage continued user interaction.
- **Installable:** Allows users to add the app to their home screen without app store downloads.
- **Offline Functionality:** Can function in low or no connectivity conditions using cached content.

Limitations:

- **Higher Battery Consumption:** PWAs tend to consume more battery due to constant background processes.
- **Limited Hardware Access:** Some device features, such as advanced sensors and Bluetooth, may not be fully accessible.
- **Offline Mode Constraints:** Some offline capabilities remain limited, depending on browser support.
- **No App Store Presence:** PWAs cannot generate traffic from app store searches.
- **Lack of Centralized Control:** Unlike native apps, PWAs do not undergo an official approval process, potentially affecting credibility.

CODE:

Index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0"
  />

    <!-- Favicon -->
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />

    <!-- Fonts -->
    <link
href="https://fonts.googleapis.com/css2?family=Poppins:wght@500&display=sw
ap" rel="stylesheet">
    <link
href="https://fonts.googleapis.com/css2?family=Satisfy&display=swap"
rel="stylesheet">

    <!-- FontAwesome Icons -->
    <script src="https://kit.fontawesome.com/25b9223bba.js"
crossorigin="anonymous"></script>

    <!-- PWA Manifest -->
    <link rel="manifest" href="/manifest.json">

    <!-- Theme Color (for PWA UI) -->
    <meta name="theme-color" content="#ffffff">

    <title>React Recipe App</title>
  </head>
  <body>
    <div id="root"></div>

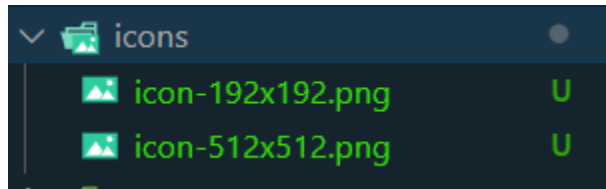
    <!-- Main Script -->
    <script type="module" src="/src/main.jsx"></script>
```

```
<!-- Register Service Worker for PWA -->
<script>
  if ('serviceWorker' in navigator) {
    window.addEventListener('load', () => {
      navigator.serviceWorker.register('/sw.js').then(registration =>
      {
        console.log('Service Worker registered:', registration);
      }).catch(error => {
        console.log('Service Worker registration failed:', error);
      });
    });
  }
</script>
</body>
</html>
```

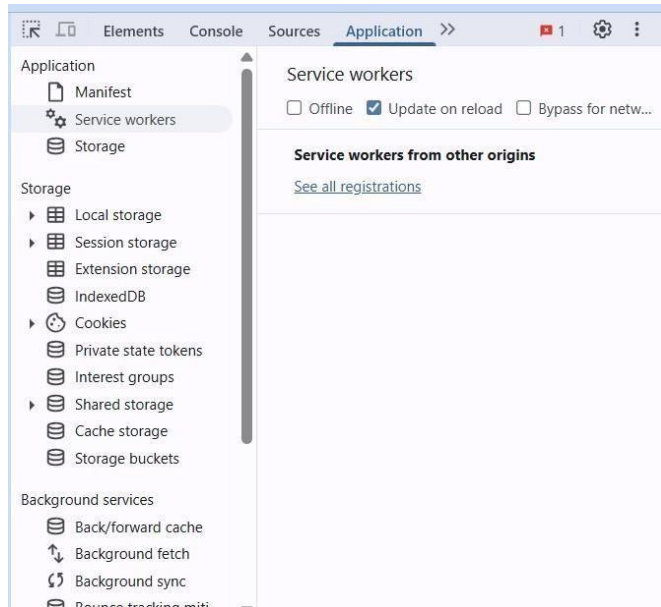
Manifest.json

```
{
  "name": "React Recipe App",
  "short_name": "RecipeApp",
  "description": "A Progressive Web App for recipes!",
  "theme_color": "#ffffff",
  "background_color": "#ffffff",
  "display": "standalone",
  "start_url": "/",
  "icons": [
    {
      "src": "/icons/icon-192x192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon-512x512.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

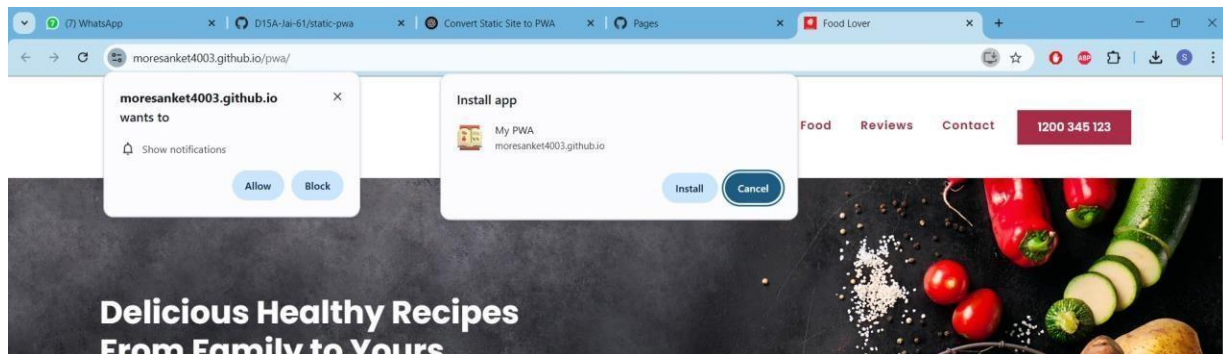
Icons

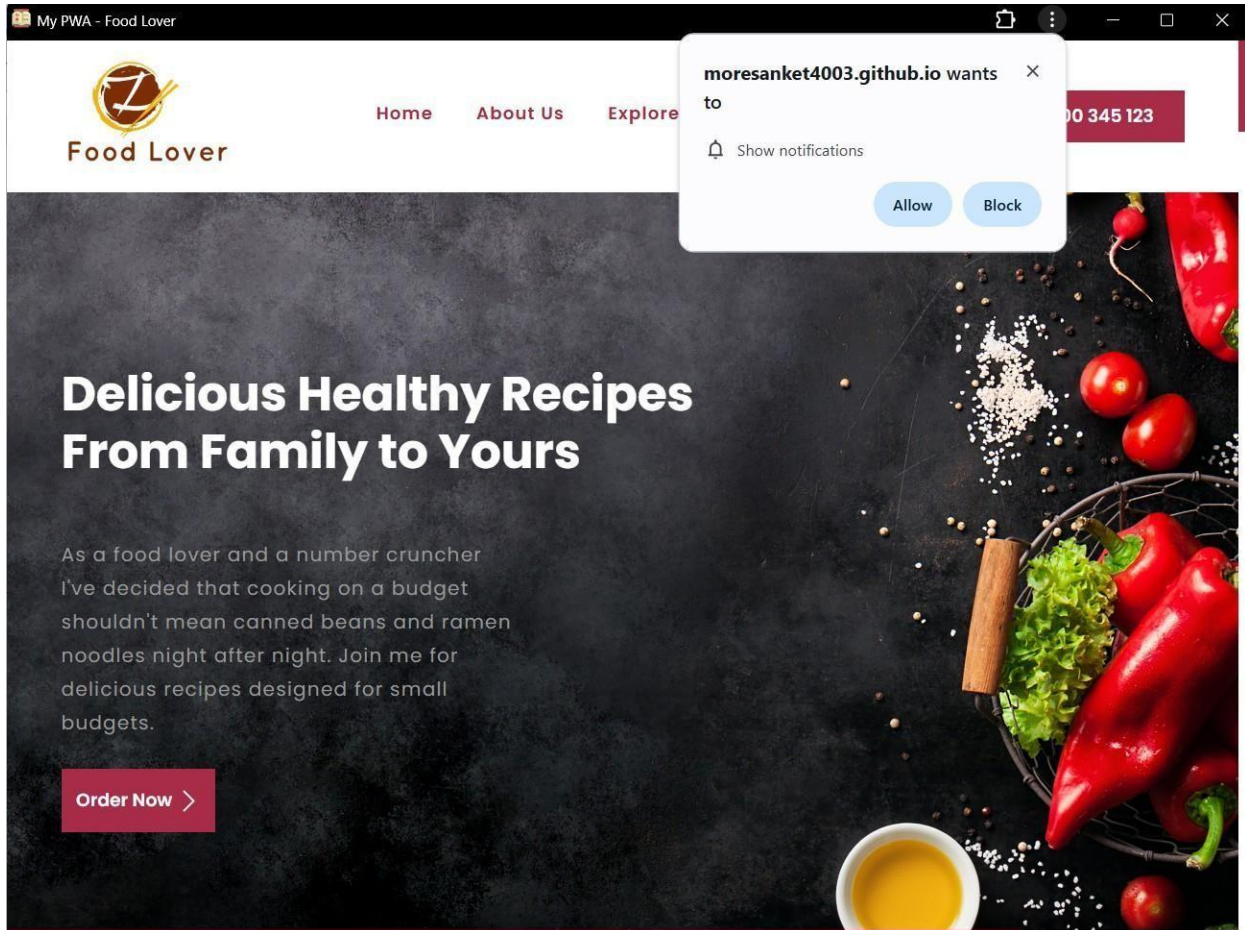


Google Dev



Output:-





Project Title: OLX(buy&sell)

Roll No. 30

MAD & PWA Lab

Journal

Experiment No.	08
Experiment Title.	To code and register a service worker, and complete the install and activation process for a new service worker for the E-commerce PWA
Roll No.	30
Name	Jai Navani
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO5: Design and Develop a responsive User Interface by applying PWA Design techniques
Grade:	

PWA Experiment -8

Jai Navani

D15A 30

Theory:-

Understanding Service Workers

A Service Worker is a JavaScript file that runs independently in the background of a browser without direct user interaction. It acts as a proxy, handling network requests, managing push notifications, and enabling offline functionalities using the Cache API.

Key Characteristics of Service Workers:

- They act as a programmable network proxy, allowing control over network requests.
- Service workers function only over HTTPS due to security concerns, preventing potential "man-in-the-middle" attacks.
- They become idle when not in use and restart as needed. To persist data across restarts, IndexedDB can be used.

Capabilities of Service Workers:

- **Network Traffic Management:** Service workers can intercept and manipulate network requests. For example, when a request is made for a CSS file, the service worker can return a plain text response instead.
- **Caching Mechanism:** They store request-response pairs, making offline access possible.
- **Push Notifications:** They enable push notifications, enhancing user engagement.
- **Background Sync:** They allow processes to continue even when there is no active internet connection.

Limitations of Service Workers:

- **No Direct Access to the Window Object:** Service workers cannot manipulate the DOM directly. However, communication with the window is possible through the `postMessage` API.
- **Must Run on HTTPS:** Service workers function only over secure connections, except when being tested on localhost.

Lifecycle of a Service Worker

A service worker undergoes three main stages:

1. **Registration**
2. **Installation**
3. **Activation**

1. Registration

To initialize a service worker, it must be registered in the main JavaScript file. This process informs the browser about the service worker location and initiates its installation.

Example: Registering a Service Worker

```
if ('serviceWorker' in navigator) {  
  navigator.serviceWorker.register('/service-worker.js')  
    .then(function(registration) {  
      console.log('Service Worker registered successfully with scope:',  
registration.scope);  
    })  
    .catch(function(error) {  
      console.log('Service Worker registration failed:', error);  
    });  
}
```

This snippet first checks if the browser supports service workers. If it does, `navigator.serviceWorker.register()` is called, returning a promise. Upon successful registration, the scope of the service worker is logged.

Defining the Scope: The scope determines which files the service worker controls. By default, it covers all files in the same directory and its subdirectories.

Setting a Custom Scope:

```
navigator.serviceWorker.register('/service-worker.js', { scope: '/app/' });
```

In this case, the service worker controls all requests under `/app/`, including subdirectories.

If the service worker needs to cover higher-level directories, the `Service-Worker-Allowed` HTTP header must be configured on the server.

2. Installation

After registration, the browser attempts to install the service worker. If the site lacks an existing service worker or if modifications are detected in the new version, the installation process is triggered.

During installation, service workers can cache essential files to enable faster subsequent page loads.

Example: Installation Event Listener

```
self.addEventListener('install', function(event) {  
  // Perform installation tasks like caching resources  
});
```

3. Activation

Once installed, the service worker moves to the activation phase. If an older version of the service worker is already in use, the new one enters a waiting state until all instances of the old service worker close.

Example: Activation Event Listener

```
self.addEventListener('activate', function(event) {  
  // Clean up outdated caches or perform setup tasks
```

```
});
```

Once activated, the new service worker takes control of pages within its scope, intercepting network requests and enabling offline functionalities. However, pages loaded before activation remain unaffected until they are refreshed or reopened.

To immediately apply the new service worker to all active pages,

`clients.claim()` can be used.

```
self.addEventListener('activate', function(event) {  
  event.waitUntil(clients.claim());  
});
```

CODE:-

Changes Made in index.html

```
<body>  
  
<script>  
  if ("serviceWorker" in navigator) {  
    navigator.serviceWorker.register("sw.js")  
      .then((reg) => {  
        console.log("Service Worker registered!", reg);  
  
        // Background Sync  
        if ("SyncManager" in window) {  
          navigator.serviceWorker.ready.then((swReg) => {  
            return swReg.sync.register("sync-data");  
          });  
        }  
  
        // Push Notifications  
        if ("PushManager" in window) {  
          Notification.requestPermission().then((permission) => {  
            if (permission === "granted") {  
              console.log("Push notifications allowed!");  
            }  
          });  
        }  
      })  
  }
```

```
        .catch((err) => console.log("Service Worker registration
failed!", err));
    }
</script>
```

sw.js

```
const CACHE_NAME = "pwa-cache-v1";
const urlsToCache = [
    "/index.html",
    "/assets/css/style.css", // Update with actual CSS filename
    "/assets/images/icon-192x192.png",
    "/assets/images/icon-512x512.png"
];

// Install event: Caches assets
self.addEventListener("install", (event) => {
    event.waitUntil(
        caches.open(CACHE_NAME).then((cache) => {
            return cache.addAll(urlsToCache);
        })
    );
});

// Fetch event: Serve cached files when offline
self.addEventListener("fetch", (event) => {
    event.respondWith(
        caches.match(event.request).then((response) => {
            return response || fetch(event.request);
        })
    );
});

// Sync event: Background sync (requires registration in main script)
self.addEventListener("sync", (event) => {
    if (event.tag === "sync-data") {
        event.waitUntil(syncDataFunction());
    }
});
```

```

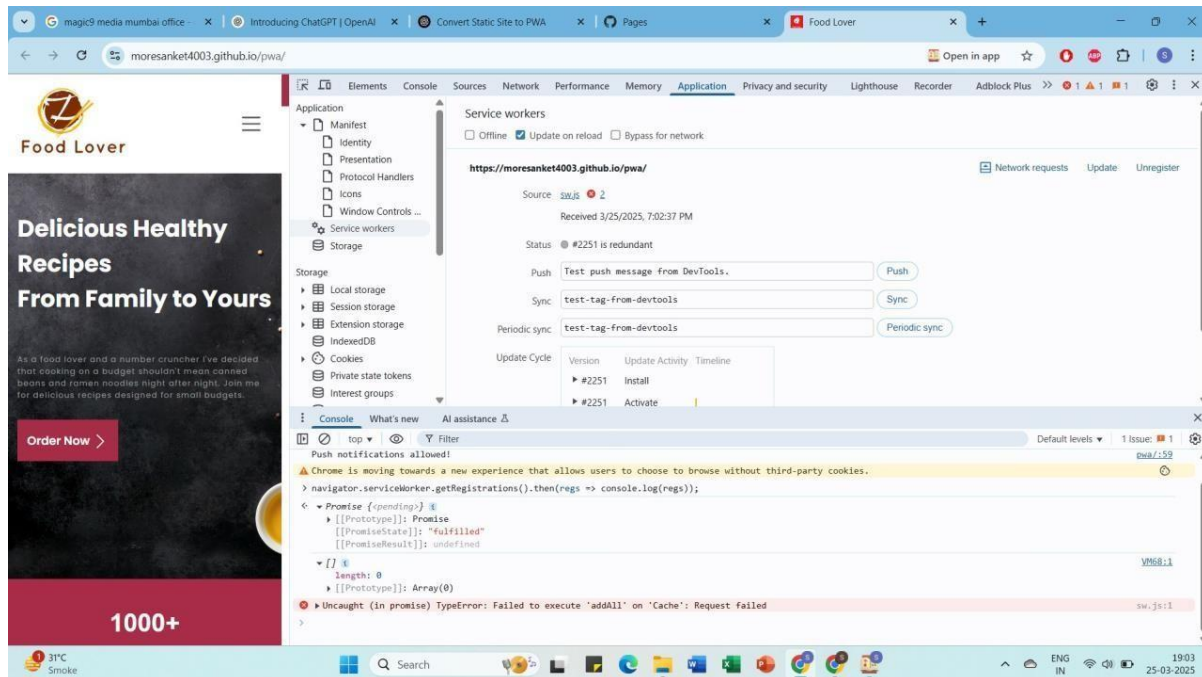
async function syncDataFunction() {
  console.log("Syncing data in the background...");
  // Example: Send stored requests to server when online
}

// Push event: Handle push notifications
self.addEventListener("push", (event) => {
  const options = {
    body: "New message received!",
    icon: "/assets/images/icon-192x192.png",
    badge: "/assets/images/icon-192x192.png"
  };
  event.waitUntil(
    self.registration.showNotification("PWA Notification", options)
  );
});

// Activate event: Clean up old caches
self.addEventListener("activate", (event) => {
  event.waitUntil(
    caches.keys().then((cacheNames) => {
      return Promise.all(
        cacheNames
          .filter((cacheName) => cacheName !== CACHE_NAME)
          .map((cacheName) => caches.delete(cacheName))
      );
    })
  );
});

```

OUTPUT:-



Project Title: OLX(buy&sell)

Roll No. 30

MAD & PWA Lab

Journal

Experiment No.	09
Experiment Title.	To implement Service worker events like fetch, sync and push for E-commerce PWA
Roll No.	30
Name	Jai Navani
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO5: Design and Develop a responsive User Interface by applying PWA Design techniques
Grade:	

PWA Experiment -9

Jai Navani

D15A 30

Aim: To implement Service worker events like fetch, sync and push for E-commerce PWA

Theory:

Service Worker

Service Worker is a script that works on browser background without user interaction independently. Also, It resembles a proxy that works on the user side. With this script, you can track network traffic of the page, manage push notifications and develop “offline first” web applications with Cache API.

Things to note about Service Worker:

- A service worker is a programmable network proxy that lets you control how network requests from your page are handled.
- Service workers only run over HTTPS. Because service workers can intercept network requests and modify responses, "man-in-the-middle" attacks could be very bad.
- The service worker becomes idle when not in use and restarts when it's next needed. You cannot rely on a global state persisting between events. If there is information that you need to persist and reuse across restarts, you can use IndexedDB databases.
- Service workers make extensive use of promises, so if you're new to promises, then you should stop reading this and check out Promises, an introduction.

Fetch Event

You can track and manage page network traffic with this event. You can check

existing cache, manage “cache first” and “network first” requests and return a response that you want.

Of course, you can use many different methods but you can find in the following example a “cache first” and “network first” approach. In this example, if the request’s and current location’s origin are the same (Static content is requested.), this is called “cacheFirst” but if you request a targeted external URL, this is called “networkFirst”.

- **CacheFirst** - In this function, if the received request has cached before, the cached response is returned to the page. But if not, a new response requested from the network.
- **NetworkFirst** - In this function, firstly we can try getting an updated response from the network, if this process completed successfully, the new response will be cached and returned. But if this process fails, we check whether the request has been cached before or not. If a cache exists, it is returned to the page, but if not, this is up to you. You can return dummy content or information messages to the page.

Sync Event

Background Sync is a Web API that is used to delay a process until the Internet connection is stable. We can adapt this definition to the real world; there is an e-mail client application that works on the browser and we want to send an email with this tool. Internet connection is broken while we are writing e-mail content and we didn’t realize it. When completing the writing, we click the send button.

Here is a job for the Background Sync.

The following view shows the classical process of sending email to us. If the Internet Connection is broken, we can’t send any content to Mail Server.

Push Event

This is the event that handles push notifications that are received from the server.

You can apply any method with received data.

We can check in the following example.

“Notification.requestPermission();” is the necessary line to show notification to the user. If you don’t want to show any notification, you don’t need this line.

In the following code block is in sw.js file. You can handle push notifications with this event. In this example, I kept it simple. We send an object that has “method” and “message” properties. If the method value is “pushMessage”, we open the information notification with the “message” property.

CODE:-

Changes Made in Index.html

```
<body>

<script>
  if ("serviceWorker" in navigator) {
    navigator.serviceWorker.register("sw.js")
      .then((reg) => {
        console.log("Service Worker registered!", reg);

        // Background Sync
        if ("SyncManager" in window) {
          navigator.serviceWorker.ready.then((swReg) => {
            return swReg.sync.register("sync-data")
              .then(() => console.log("Sync event registered"))
              .catch((err) => console.log("Sync registration
failed", err));
          });
        }

        // Push Notifications
        if ("PushManager" in window) {
          Notification.requestPermission().then((permission) => {
            if (permission === "granted") {
```

```

        console.log("Push notifications allowed!");
    } else {
        console.log("Push notifications denied!");
    }
    });
    }
    })
    .catch((err) => console.log("Service Worker registration
failed!", err));
    }

    // Function to send a message to the Service Worker
    function sendMessageToServiceWorker(action) {
        if (navigator.serviceWorker.controller) {
            navigator.serviceWorker.controller.postMessage({ action:
action });
        } else {
            console.log("Service Worker is not active yet.");
        }
    }

    // Detect clicks on "Contact Us" navigation link
    document.getElementById("contactUs")?.addEventListener("click",
() => {
        sendMessageToServiceWorker("contact_us_nav_clicked");
    });

    // Detect clicks on "Contact Us" button
    document.getElementById("contactUsBtn")?.addEventListener("click", () =>
{
        sendMessageToServiceWorker("contact_us_btn_clicked");
    });
</script>

<!-- **** Header Section **** -->

<header>
    <nav class="navbar navbar-expand-lg navigation-wrap">

```

```

        <div class="container">
            <a class="navbar-brand" href="#">
                
            </a>
            <button class="navbar-toggler" type="button"
data-bs-toggle="collapse" data-bs-target="#navbarNav"
                aria-controls="navbarNav" aria-expanded="false"
aria-label="Toggle navigation">
                <span class="navbar-toggler-icon"></span>
            </button>
            <div class="collapse navbar-collapse" id="navbarNav">
                <ul class="navbar-nav ms-auto">
                    <li class="nav-item">
                        <a class="nav-link" href="#home">Home</a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link" href="#about">About
Us</a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link"
href="#explore-food">Explore Food</a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link"
href="#testinomial">Reviews</a>
                    </li>

                    <!-- Contact Us Nav Link -->
                    <li class="nav-item">
                        <a class="nav-link" href="#contact"
id="contactUs">Contact</a>
                    </li>

                    <!-- Contact Us Button -->
                    <li>
                        <button class="main-btn"
id="contactUsBtn">1200 345 123</button>
                    </li>
                </ul>
            </div>
        </div>

```

```

        </ul>
      </div>
    </div>
  </nav>
</header>

```

New sw.js

```

const CACHE_NAME = "pwa-cache-v1";
const urlsToCache = [
  "/index.html",
  "/assets/css/style.css", // Update with actual CSS filename
  "/assets/images/icon-192x192.png",
  "/assets/images/icon-512x512.png"
];

// Install event: Caches assets
self.addEventListener("install", (event) => {
  event.waitUntil(
    caches.open(CACHE_NAME).then((cache) => {
      return cache.addAll(urlsToCache);
    })
  );
});

// Fetch event: Serve cached files when offline
self.addEventListener("fetch", (event) => {
  event.respondWith(
    caches.match(event.request).then((response) => {
      return response || fetch(event.request);
    })
  );
});

// Sync event: Background sync (requires registration in main script)
self.addEventListener("sync", (event) => {
  if (event.tag === "sync-data") {
    event.waitUntil(syncDataFunction());
  }
});

```

```

    }
  });

  async function syncDataFunction() {
    console.log("Syncing data in the background...");
    // Example: Send stored requests to server when online
  }

  // Push event: Handle push notifications
  self.addEventListener("push", (event) => {
    const options = {
      body: "New message received!",
      icon: "/assets/images/icon-192x192.png",
      badge: "/assets/images/icon-192x192.png"
    };
    event.waitUntil(
      self.registration.showNotification("PWA Notification", options)
    );
  });

  // Message event: Handle messages from the main thread
  self.addEventListener("message", (event) => {
    if (event.data && event.data.action) {
      console.log(`Service Worker: User clicked on ${event.data.action}`);

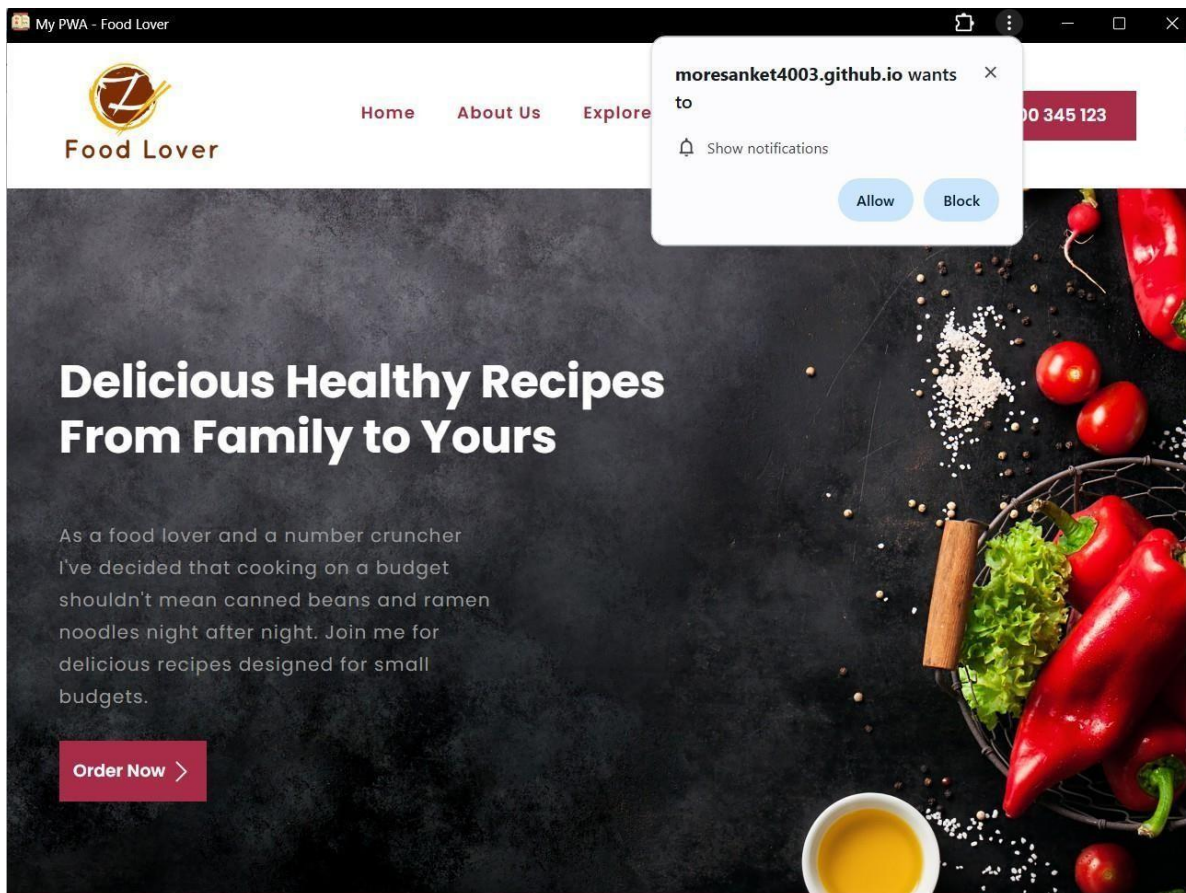
      // Example: Show a notification when "Contact Us" is clicked
      if (event.data.action === "contact_us_nav_clicked" ||
event.data.action === "contact_us_btn_clicked") {
        self.registration.showNotification("Contact Us Clicked", {
          body: "User clicked the Contact Us button!",
          icon: "/assets/images/icon-192x192.png"
        });
      }
    }
  });

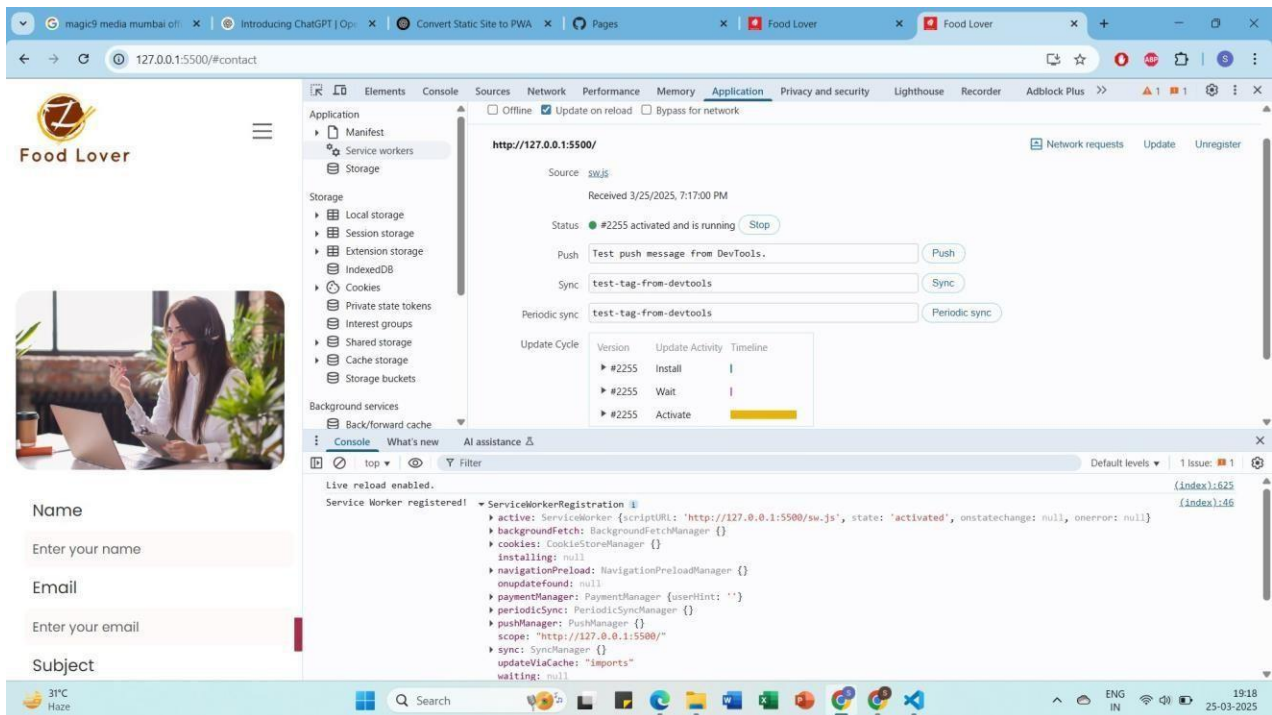
  // Activate event: Clean up old caches
  self.addEventListener("activate", (event) => {
    event.waitUntil(
      caches.keys().then((cacheNames) => {

```

```
return Promise.all(  
  cacheNames  
    .filter((cacheName) => cacheName !== CACHE_NAME)  
    .map((cacheName) => caches.delete(cacheName))  
  );  
})  
);  
});
```

OUTPUT:-





```
waiting: null
```

```
▶ [[Prototype]]: ServiceWorkerRegistration
```

```
Sync event registered
```

```
Syncing data in the background...
```

```
Live reload enabled.
```

```
Service Worker registered! ▶ ServiceWorkerRegistration
```

```
Sync event registered
```

```
Syncing data in the background...
```

```
Push notifications allowed!
```

MAD & PWA Lab

Journal

Experiment No.	10
Experiment Title.	To study and implement deployment of Ecommerce PWA to GitHub Pages.
Roll No.	30
Name	Jai Navani
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO5: Design and Develop a responsive User Interface by applying PWA Design techniques
Grade:	

PWA Experiment -10

Jai Navani

D15A 30

Aim:

To study and implement deployment of Ecommerce PWA to GitHub Pages.

Theory:

GitHub Pages

Public web pages are freely hosted and easily published. Public webpages hosted directly from your GitHub repository. Just edit, push, and your changes are live.

GitHub Pages provides the following key features:

1. Blogging with Jekyll
2. Custom URL
3. Automatic Page Generator

Reasons for favoring this over Firebase:

1. Free to use
2. Right out of github
3. Quick to set up

GitHub Pages is used by Lyft, CircleCI, and HubSpot.

GitHub Pages is listed in 775 company stacks and 4401 developer stacks.

Pros

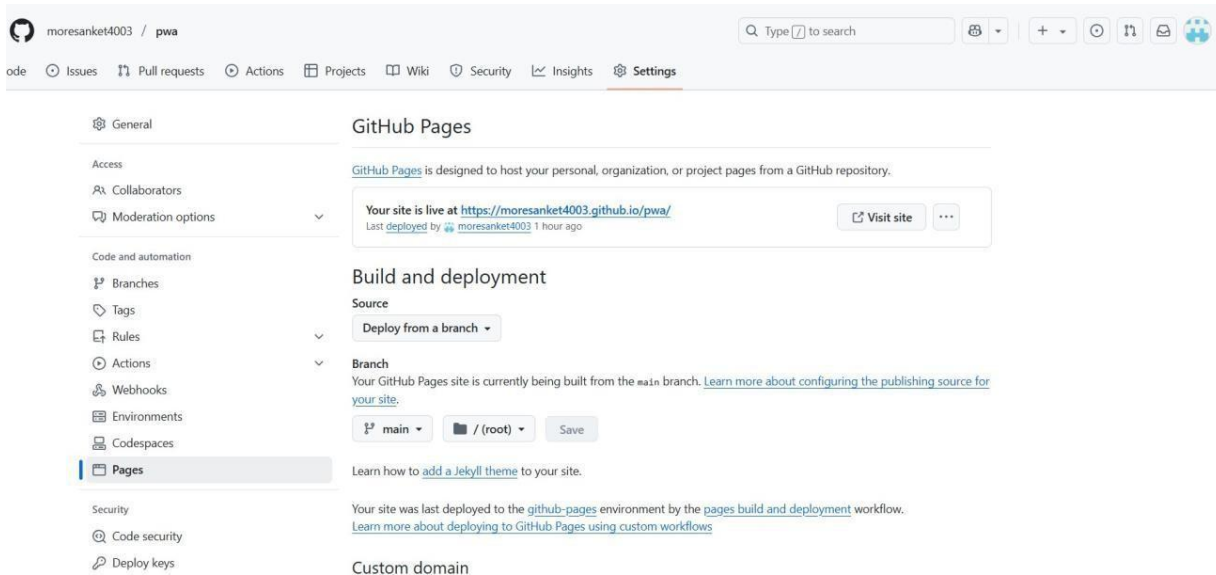
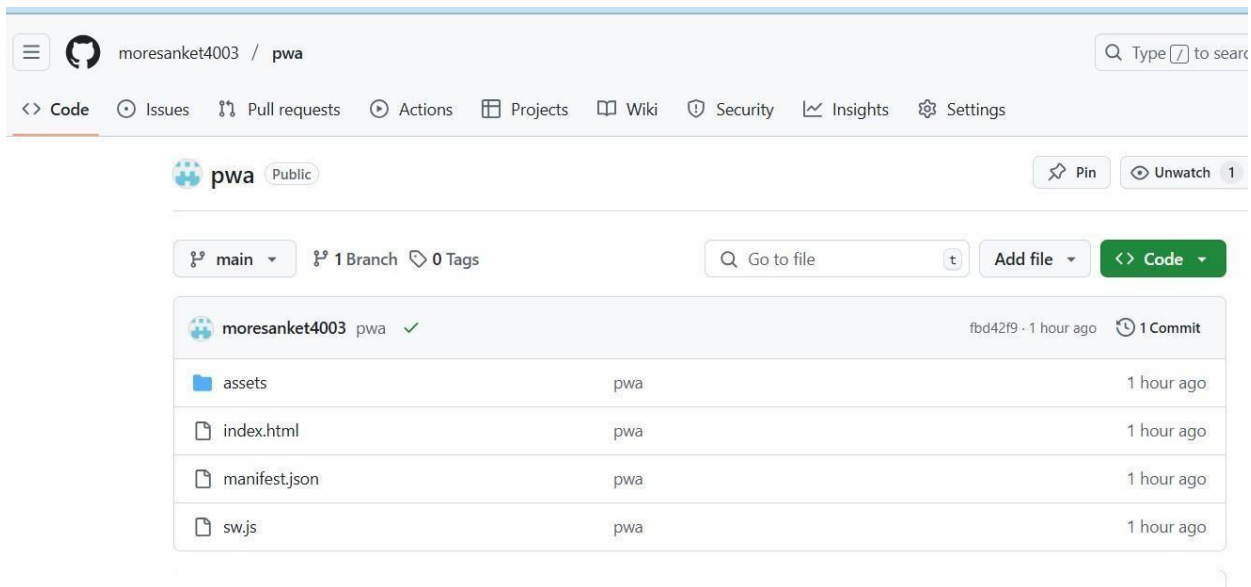
1. Very familiar interface if you are already using GitHub for your projects.
2. Easy to set up. Just push your static website to the gh-pages branch and your website is ready.
3. Supports Jekyll out of the box.
4. Supports custom domains. Just add a file called CNAME to the root of your site, add an A record in the site's DNS configuration, and you are done.

Cons

1. The code of your website will be public, unless you pay for a private repository.
2. Currently, there is no support for HTTPS for custom domains. It's probably coming soon though.
3. Although Jekyll is supported, plug-in support is rather spotty.

[Link to our GitHub repository:](#)

OUTPUT:-



MAD & PWA Lab

Journal

Experiment No.	11
Experiment Title.	To use google Lighthouse PWA Analysis Tool to test the PWA functioning.
Roll No.	30
Name	Jai Navani
Class	D15A/D15B
Subject	MAD & PWA Lab
Lab Outcome	LO6: Develop and Analyze PWA Features and deploy it over app hosting solution
Grade:	

PWA Experiment 11

Jai Navani

D15A 30

Aim : To use google Lighthouse PWA Analysis Tool to test the PWA functioning.

Theory :

Google Lighthouse :

Google Lighthouse is a tool that lets you audit your web application based on a number of parameters including (but not limited to) performance, based on a number of metrics, mobile compatibility, Progressive Web App (PWA) implementations, etc. All you have to do is run it on a page or pass it a URL, sit back for a couple of minutes and get a very elaborate report, not much short of one that a professional auditor would have compiled in about a week.

The best part is that you have to set up almost nothing to get started. Let's begin by looking at some of the top features and audit criteria used by Lighthouse.

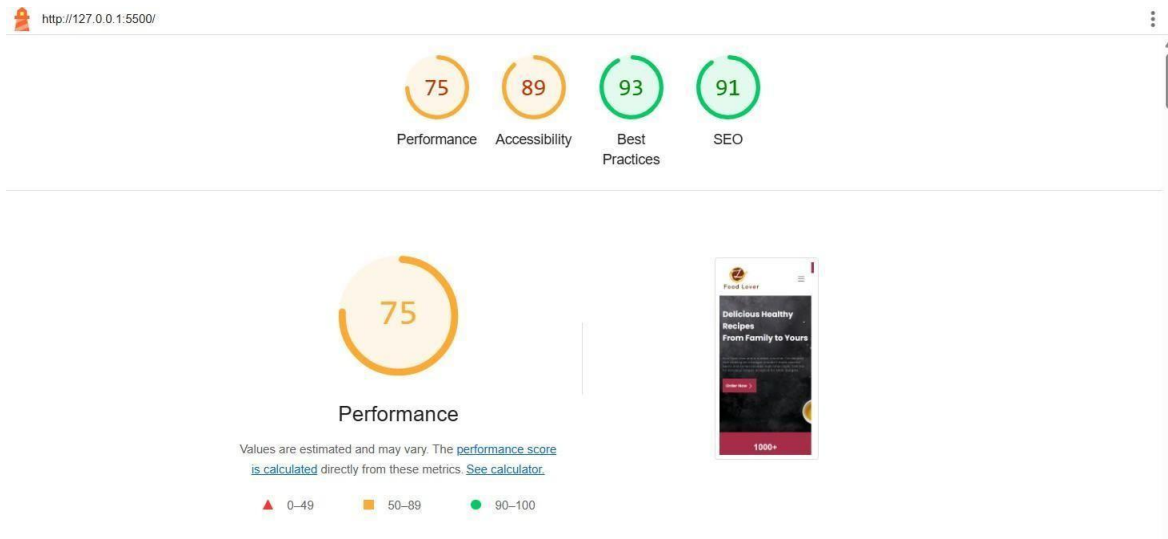
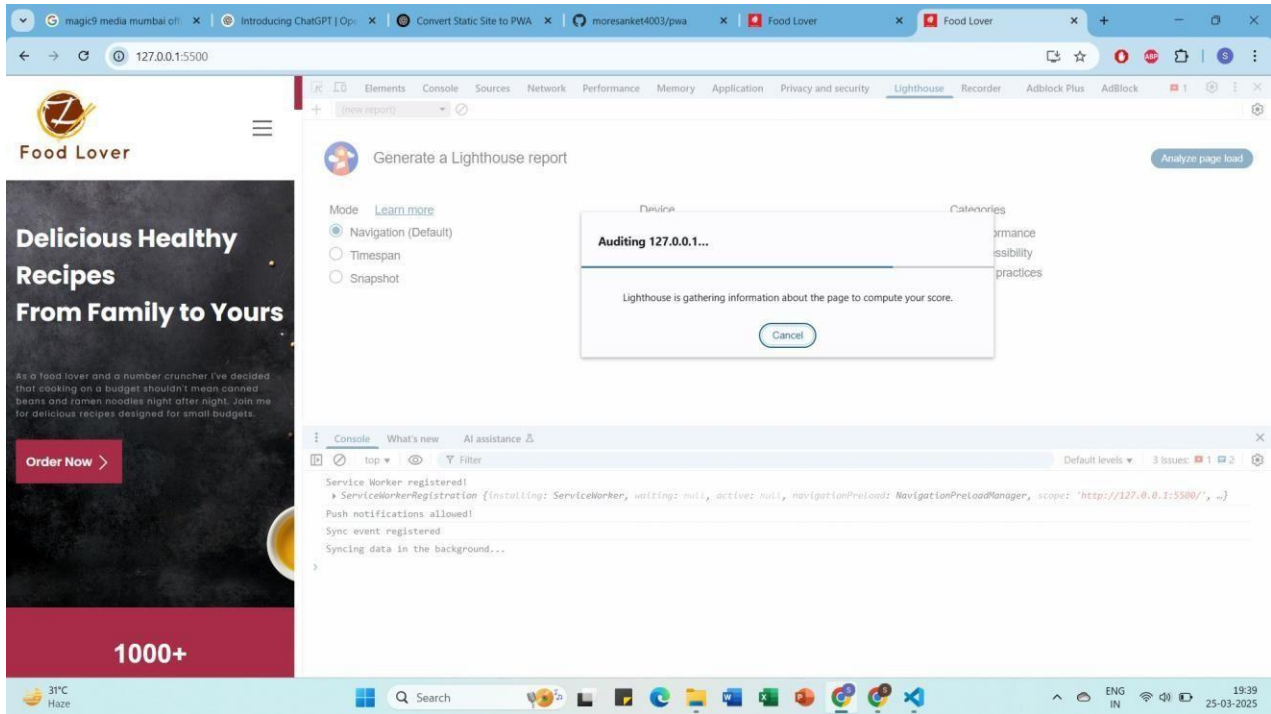
Key Features and Audit Metrics

Google Lighthouse has the option of running the Audit for Desktop as well as mobile version of your page(s). The top metrics that will be measured in the Audit are:

1. **Performance:** This score is an aggregation of how the page fared in aspects such as (but not limited to) loading speed, time taken for loading for basic frame(s), displaying meaningful content to the user, etc. To a layman, this score is indicative of how decently the site performs, with a score of 100 meaning that you figure in the 98th percentile, 50 meaning that you figure in the 75th percentile and so on.

2. **PWA Score (Mobile):** Thanks to the rise of Service Workers, app manifests, etc., a lot of modern web applications are moving towards the PWA paradigm, where the objective is to make the application behave as close as possible to native mobile applications. Scoring points are based on the Baseline PWA checklist laid down by Google which includes Service Worker implementation(s), viewport handling, offline functionality, performance in script-disabled environments, etc.
3. **Accessibility:** As you might have guessed, this metric is a measure of how accessible your website is, across a plethora of accessibility features that can be implemented in your page (such as the 'aria-' attributes like aria-required, audio captions, button names, etc.). Unlike the other metrics though, Accessibility metrics score on a pass/fail basis i.e. if all possible elements of the page are not screen-reader friendly (HTML5 introduced features that would make pages easy to interpret for screen readers used by visually challenged people like tag names, tags such as <section>, <article>, etc.), you get a 0 on that score. The aggregate of these scores is your Accessibility metric score.
4. **Best Practices:** As any developer would know, there are a number of practices that have been deemed 'best' based on empirical data. This metric is an aggregation of many such points, including but not limited to:
 - Use of HTTPS
 - Avoiding the use of deprecated code elements like tags, directives, libraries, etc.
 - Password input with paste-into disabled
 - Geo-Location and cookie usage alerts on load, etc.

OUTPUT:-





Best Practices

TRUST AND SAFETY

Conclusion: Thus we successfully used google Lighthouse PWA Analysis Tool for testing the PWA functioning.